My SBA Report
2026 - 2027

English Learning Platform

6T 17 Tong Chi Ching Isaac

Design and Implementation Vision

Our Objectives

- **Primary Goal**: Develop an effective, user-friendly English learning platform with interactive content.
- **Engagement**: Feature an intuitive UI to boost user motivation.
- **Flexibility**: Offer three difficulty modes tailored to different proficiency levels.
- **Tech Stack**: Built with Python to ensure system flexibility and efficiency.
- **Impact**: Support diverse learners, encourage active participation, and enhance language skills

My Own Personal Goal

- **By doing this project I wish to**:
  - further improve my own programming skill
  - Increase my English vocabulary
  - Help other people who’s English level are low
  - Encourage people to learn english

My Main Function

- Login
- Creating a new account
- Dashboard
- Change UI theme
- Play Games
- Leaderboard
- Profile change related
- Logout / Exit

Program code

For my program code , it is split into 3 **Main** section:

- Background Code
- UI code
- Database Code

Background Code

(Define function to launch app and Define function for each UI button)

Program Code (Main/Start Up Code)

```
import argparse
from pathlib import Path
import sqlite3
import tkinter as tk
from tkinter import messagebox, ttk
```

```
from collections import Counter
import random
```

- **Imports argparse library**: Loads CLI argument parser.
- **Imports pathlib Path**: Loads file path utility.
- **Imports sqlite3**: Loads database module.
- **Imports tkinter**: Loads GUI toolkit.
- **Imports messagebox and ttk**: Loads dialogs and themed widgets.
- **Imports `Counter`**: Loads frequency counting tool
- **Imports `random`**: Loads random utility module

```
def main():
    parser = argparse.ArgumentParser(description="Play three English word games with SQLite leaderboards.")
    parser.add_argument("--database", type=Path, help="Use a different SQLite database (default: beside main.py).")
    args = parser.parse_args()
    root = tk.Tk()
    try:
        database = Database(args.database) if args.database else Database()
    except (OSError, ValueError, sqlite3.Error) as error:
        root.withdraw()
        messagebox.showerror("Database could not be opened", str(error), parent=root)
        root.destroy()
        return
    EnglishLearningApp(root, database)
    report = database.migration_report
    if any(report[key] for key in ("users_imported", "questions_imported", "questions_archived",
                                   "users_skipped", "questions_skipped")) or report["warnings"]:
        details = (f"Imported {report['users_imported']} accounts and {report['questions_imported']} questions.\n"
                   f"Preserved {report['legacy_scores_imported']} old cumulative scores separately from the new leaderboards.\n"
                   f"Archived {report['questions_archived']} questions that need a fill-in-the-blank rewrite.\n\n"
                   "Original text files have been left unchanged. Each file is imported once.")
        if report["warnings"]:
            details += "\n\n" + "\n".join(report["warnings"][:6])
            if len(report["warnings"]) > 6:
                details += f"\n...and {len(report['warnings']) - 6} more import notices."
        root.after_idle(lambda: messagebox.showinfo("Previous data imported", details, parent=root))
    root.mainloop()
```

- **Parse Arguments**: Processes optional/custom `database` file path
- **Initialize GUI**: Creates the root Tkinter window
- **Connect Database**: Opens SQLite database;exit if fail
- **Launch App**: Starts the main `EnglishLearningApp`
- **Migration Report**: Checks for database imports/warnings
- **Main Loop**: Starts The GUI Execution Loop

Program Code (Main Code)

```python
def __init__(self, root, database=None):
    self.root = root
    self.db = database if database is not None else Database()
    self.theme_name = self.db.get_theme()
    self.colors = THEMES[self.theme_name]
    root._theme_colors = self.colors
    self._feedback_status = "info"
    self.active_user = None
    self.session = None
    self.best_score = self.starting_best = 0
    self.frames = []
    self.current_frame = None
    self.auth_mode = tk.StringVar(root, value="login")
    self.leaderboard_mode = tk.StringVar(root, value="easy")
    self.guess_var = tk.StringVar(root)
    self._game_is_wide = None
    self._closing = False
    root.title("English Learner")
    root.configure(bg=self.colors["bg"])
    width = min(1080, root.winfo_screenwidth() - 60)
    height = min(850, root.winfo_screenheight() - 90)
    root.geometry(f"{width}x{height}")
    root.minsize(760, 640)
    root.protocol("WM_DELETE_WINDOW", self.close)
    configure_styles(root)
    self._set_window_icon()
    self.setup_shell()
    self.setup_login_screen()
    self.setup_menu_screen()
    self.setup_mode_screen()
    self.setup_game_screen()
    self.setup_end_screen()
    self.setup_leaderboard_screen()
    self.setup_edit_profile_screen()
    self.guess_var.trace_add("write", self._on_guess_changed)
    root.bind("<Return>", self._on_enter)
    root.bind("<KP_Enter>", self._on_enter)
    root.bind("<MouseWheel>", self._on_mousewheel)
    root.bind("<Button-4>", lambda event: self._on_mousewheel(event, -3))
    root.bind("<Button-5>", lambda event: self._on_mousewheel(event, 3))
    root.bind("<FocusIn>", self._on_focus)
    root.bind("<Control-a>", self._select_all)
    self.show_screen(self.frame_login)
    self.entry_username.focus_set()
```

- **Initializes root and database**: Sets core references
- **Configures themes and variables**: Loads state and colors
- **Sets window geometry**: Defines dimensions and constraints
- **Executes UI setup methods**: Builds all screen layouts
- **Binds input events**: Attaches keyboard and mouse
- **Displays initial screen**: Shows login with focus

Program Code (Login/Create Account Screen Code)

```
def setup_login_screen(self):
    self.frame_login = self._page()
    self.frame_login.max_width = 640
    body = self.frame_login.body
    self._heading(body, "English Learner")
    form = card(body, padding=24)
    form.pack(fill="x")
    tabs = tk.Frame(form, bg=self.colors["surface"])
    tabs.pack(fill="x", pady=(0, 12))
    self.auth_tabs = {}
    for mode, text in (("login", "Log in"), ("register", "Create account")):
        button = ActionButton(tabs, text=text, variant="primary" if mode == "login" else "secondary",
                              command=lambda m=mode: self.set_auth_mode(m))
        button.pack(side="left", fill="x", expand=True, padx=(0, 8) if mode == "login" else 0)
        self.auth_tabs[mode] = button
    self.entry_username, _ = self._field(form, "Username")
    self.entry_password, self.btn_show_password = self._field(form, "Password", password=True)
    self.auth_feedback = label(form, "", color=self.colors["muted"], wraplength=780, justify="left")
    self.btn_auth = ActionButton(form, text="Log in", command=self.submit_auth)
    self.btn_auth.pack(fill="x", pady=(20, 0))
```

Function:

- **Create Login Frame**
- **Add Header**:"English Learner"
- **Setup Form Card**: Creates a padded container card
- **Create Tabs**: Adds tab buttons to switch mode
- **Add Input Fields**: username and password fields
- **Add Feedback Label**: Show success/error messages
- **Add Submit Button**

Program Code (Create/Login Account Related Code Part 1)

```python
def set_auth_mode(self, mode):
    if mode not in ("login", "register"):
        return
    self.auth_mode.set(mode)
    registering = mode == "register"
    self.btn_auth.config(text="Create account" if registering else "Log in")
    self._form_message(self.auth_feedback, "")
    for key, button in self.auth_tabs.items():
        button.config(variant="primary" if key == mode else "secondary")
    self.entry_password.delete(0, tk.END)
    self._hide_passwords()
    self.entry_username.focus_set()

def submit_auth(self):
    if self.auth_mode.get() == "register":
        self.register()
    else:
        self.login()
```

`set_auth_mode` Function:

- **Validate mode**: Checks mode validity
- **Update state**: Sets auth mode
- **Configure button**: Updates button text
- **Clear message**: Resets feedback
- **Update tabs**: Highlights active tab
- **Reset fields**: Clears password and focuses username

`submit_auth` Function:

- **Route action**: Calls register or login

Program Code (Create/Login Account Related Code Part 2)

```python
def register(self):
    try:
        self.db.create_user(self.entry_username.get().strip(), self.entry_password.get())
    except (ValueError, sqlite3.Error) as error:
        self._form_message(self.auth_feedback, str(error), error=True)
        return
    self.set_auth_mode("login")
    self._form_message(self.auth_feedback, "Account created. Enter your password to log in.")
    self.entry_password.focus_set()
```

```python
def login(self):
    username, password = self.entry_username.get().strip(), self.entry_password.get()
    if not username or not password:
        self._form_message(self.auth_feedback, "Enter your username and password to continue.", error=True)
        (self.entry_username if not username else self.entry_password).focus_set()
        return
    try:
        user = self.db.authenticate(username, password)
    except (ValueError, sqlite3.Error) as error:
        self._form_message(self.auth_feedback, str(error), error=True)
        return
    if user is None:
        self._form_message(self.auth_feedback, "That username and password do not match. Please try again.", error=True)
        self.entry_password.focus_set()
        self.entry_password.selection_range(0, tk.END)
        return
    self.active_user = user
    self.entry_password.delete(0, tk.END)
    self._hide_passwords()
    self._form_message(self.notice, "")
    self.show_menu()
```

`register` Function:

- **Create user**: Registers new account from input fields
- **Handle error**: Displays error message on failure
- **Switch mode**: Changes view to login mode
- **Success message**: Shows account creation notice
- **Focus input**: Sets focus to password entry

`login` Function:

- **Validate input**: Checks for empty username or password
- **Authenticate**: Verifies credentials against database
- **Handle failure**: Shows error if user is invalid
- **Set active user**: Stores logged-in user state
- **Reset fields**: Clears password and hides visibility
- **Show menu**: Navigates to main dashboard

Program Code (Create/Login Account Related Code Part 3)

```python
def toggle_password(self, entry, button):
    visible = bool(entry.cget("show"))
    entry.config(show="" if visible else "*")
    button.config(text="Hide" if visible else "Show")
    entry.focus_set()

def _hide_passwords(self):
    for entry, button in ((self.entry_password, self.btn_show_password),
                          (self.entry_new_password, self.btn_show_new_password)):
        entry.config(show="*")
        button.config(text="Show")

def set_auth_mode(self, mode):
    if mode not in ("login", "register"):
        return
    self.auth_mode.set(mode)
    registering = mode == "register"
    self.btn_auth.config(text="Create account" if registering else "Log in")
    self._form_message(self.auth_feedback, "")
    for key, button in self.auth_tabs.items():
        button.config(variant="primary" if key == mode else "secondary")
    self.entry_password.delete(0, tk.END)
    self._hide_passwords()
    self.entry_username.focus_set()

def submit_auth(self):
    if self.auth_mode.get() == "register":
        self.register()
    else:
        self.login()
```

`Toggle_password Function`:

- **Toggles password field visibility**: Switches mask and button text

`_hide_passwords Function`:

- **Resets password masking**: Masks fields and updates buttons

`set_auth_mode Function`:

- **Switches auth mode**: Updates UI for login or register

`submit_auth Function`:

- **Submits authentication form**: Triggers login or register action

Program Code (Main Menu: Tabs Code)

```
def setup_shell(self):
    self.topbar = tk.Frame(self.root, bg=self.colors["bg"], padx=16, pady=8)
    self.topbar.pack(fill="x")
    self.btn_theme = ActionButton(
        self.topbar, text="Dark mode" if self.theme_name == "light" else "Light mode",
        variant="secondary", compact=True, command=self.toggle_theme)
    self.btn_theme.pack(side="right")
    self.navigation = tk.Frame(self.topbar, bg=self.colors["bg"])
    self.nav_buttons = {}
    for route, text in (("dashboard", "Dashboard"), ("play", "Play"),
                        ("leaderboard", "Leaderboards"), ("profile", "Profile")):
        button = ActionButton(self.navigation, text=text, variant="ghost", compact=True,
                              command=lambda r=route: self._navigate(r))
        button.pack(side="left", padx=2)
        self.nav_buttons[route] = button
    ActionButton(self.navigation, text="Log out", variant="ghost", compact=True,
                 command=lambda: self._navigate("logout")).pack(side="left", padx=2)
    self.content = tk.Frame(self.root, bg=self.colors["bg"])
    self.content.pack(fill="both", expand=True)
```

Function:

- **Creates topbar frame**: Sets header container
- **Adds theme button**: Creates mode switch
- **Builds navigation bar**: Sets nav container
- **Loops route buttons**: Creates main nav items
- **Adds logout button**: Creates session button
- **Creates content frame**: Sets body container

Program Code (Main Menu: Change Screen Related Code)

```python
def show_screen(self, frame):
    for other in self.frames:
        other.pack_forget()
    self.current_frame = frame
    if self.active_user is not None and frame is not self.frame_game:
        self.navigation.pack(side="left")
    else:
        self.navigation.pack_forget()
    selected = {self.frame_menu: "dashboard", self.frame_modes: "play",
                self.frame_leaderboard: "leaderboard", self.frame_edit_profile: "profile",
                self.frame_end: "play"}.get(frame)
    for route, button in self.nav_buttons.items():
        button.config(variant="secondary" if route == selected else "ghost")
    frame.pack(fill="both", expand=True)
    frame.reset_scroll()
    focus = {self.frame_login: self.entry_username, self.frame_menu: self.btn_play,
             self.frame_modes: self.mode_buttons["easy"],
             self.frame_game: self.entry_guess, self.frame_end: self.btn_replay,
             self.frame_edit_profile: self.entry_new_username,
             self.frame_leaderboard: self.leaderboard_buttons[self.leaderboard_mode.get()]}.get(frame)
    (focus or self.nav_buttons["play"]).focus_set()

def _navigate(self, route):
    routes = {"dashboard": self.show_menu, "play": self.show_mode_selection,
              "leaderboard": self.show_leaderboard, "profile": self.show_edit_profile,
              "logout": self.logout}
    if route not in routes or self.active_user is None or not self._persist_score():
        return
    routes[route]()
    if self.current_frame is not self.frame_game and self.current_frame is not self.frame_end:
        self.session = None
```

`show_screen` Function:

- **Hide frames**: Clears all existing screens
- **Toggle navigation**: Shows or hides navigation bar
- **Update styles**: Highlights active nav button
- **Render frame**: Packs target frame and resets scroll
- **Set focus**: Focuses default widget for frame

`_navigate` Function:

- **Validate route**: Checks route validity and user state
- **Persist score**: Saves score before switching
- **Execute route**: Calls corresponding screen function
- **Clear session**: Resets session if leaving game

Program Code (Main Menu: Screen Related Code)

```python
def _page(self):
    page = ScrollPage(self.content, max_width=1000, padding=24)
    self.frames.append(page)
    return page

def _heading(self, parent, title):
    title_label = label(parent, title, size=26, bold=True, wraplength=850, justify="left", anchor="w")
    title_label.pack(anchor="w", fill="x", pady=(0, 22))
    return title_label

def _field(self, parent, title, *, password=False, variable=None, hint=""):
    label(parent, title, size=10, bold=True).pack(anchor="w", pady=(12, 6))
    outline = tk.Frame(parent, bg=self.colors["surface"], highlightthickness=1, highlightbackground=self.colors["border"])
    outline.pack(fill="x")
    entry = tk.Entry(outline, textvariable=variable, show="*" if password else "", bg=self.colors["surface"], fg=self.colors["ink"],
                     font=("Segoe UI", 12), relief="flat", bd=0, insertbackground=self.colors["primary"],
                     selectbackground=self.colors["primary"], selectforeground=self.colors["on_primary"])
    entry.pack(side="left", fill="x", expand=True, padx=12, ipady=11)
    entry.bind("<FocusIn>", lambda event: outline.config(highlightbackground=self.colors["primary"]))
    entry.bind("<FocusOut>", lambda event: outline.config(highlightbackground=self.colors["border"]))
    reveal = None
    if password:
        reveal = ActionButton(outline, text="Show", variant="ghost", compact=True,
                              command=lambda: self.toggle_password(entry, reveal))
        reveal.pack(side="right", padx=4, pady=3)
    if hint:
        label(parent, hint, size=9, color=self.colors["muted"], wraplength=600, justify="left").pack(anchor="w", fill="x", pady=(5, 0))
    return entry, reveal
```

`_page` Function:

- **Create page**: Builds scrollable widget
- **Track frame**: Appends to frames list

`_heading` Function:

- **Create title**: Builds styled heading
- **Pack element**: Positions header label

`_field` Function:

- **Create label**: Renders title text
- **Build entry**: Creates input box
- **Add toggle**: Renders password show button
- **Render hint**: Displays help text

Program Code (Main Menu Tabs: Dashboard Screen Code)

```
def setup_menu_screen(self):
    self.frame_menu = self._page()
    body = self.frame_menu.body
    self.lbl_welcome = self._heading(body, "Dashboard")
    self.notice = label(body, "", color=self.colors["success"], wraplength=850, justify="left")
    self.btn_play = ActionButton(body, text="Play a Game", command=self.show_mode_selection)
    self.btn_play.pack(anchor="w", pady=(0, 28))
    label(body, "Best scores", size=15, bold=True).pack(anchor="w", pady=(0, 12))
    row = tk.Frame(body, bg=self.colors["bg"])
    row.pack(fill="x")
    self.best_value_labels = {}
    for column, (mode, (difficulty, name, _)) in enumerate(MODES.items()):
        row.columnconfigure(column, weight=1, uniform="scores")
        box = card(row, padding=18)
        box.grid(row=0, column=column, sticky="nsew", padx=(0, 12) if column < 2 else 0)
        label(box, difficulty, size=10, color=self.colors["mode_" + mode]).pack(anchor="w")
        label(box, name, size=12, bold=True, wraplength=260, justify="left", anchor="w").pack(anchor="w", fill="x", pady=(5, 14))
        self.best_value_labels[mode] = label(box, "0", size=34, bold=True)
        self.best_value_labels[mode].pack(anchor="w")
```

Function:

- **Create Menu Frame**
- **Add Header**: Adds "Dashboard" heading
- **Creates Play Button**: Action button linked to "play" Tab
- **Best Scores Section**: Add "Best scores" section container
- **Card Details**: displays difficulty level, game's name, highest score

Program Code (Main Menu Tabs: "Play" Screen Code)

```
def setup_mode_screen(self):
    self.frame_modes = self._page()
    body = self.frame_modes.body
    self._heading(body, "Choose a game")
    self.mode_buttons = {}
    for mode, (difficulty, name, rules) in MODES.items():
        box = card(body, padding=20)
        box.pack(fill="x", pady=(0, 14))
        top = tk.Frame(box, bg=self.colors["surface"])
        top.pack(fill="x")
        button = ActionButton(top, text=f"Play {difficulty}", command=lambda m=mode: self.start_game(m))
        button.pack(side="right", padx=(14, 0))
        self.mode_buttons[mode] = button
        info = tk.Frame(top, bg=self.colors["surface"])
        info.pack(side="left", fill="x", expand=True)
        label(info, difficulty, size=10, bold=True, color=self.colors["mode_" + mode]).pack(anchor="w")
        label(info, name, size=19, bold=True, wraplength=500, justify="left").pack(anchor="w", fill="x", pady=(3, 0))
        label(box, rules, color=self.colors["muted"], wraplength=850, justify="left").pack(anchor="w", fill="x", pady=(12, 0))
```

Function:

- **Initializes mode page**: Creates the game selection container
- **Adds page header**: Displays "Choose a game"
- **Loops game modes**: Iterates through available games
- **Creates game card**: Builds UI card container
- **Adds play button**: Creates button starting game
- **Displays game details**: Shows difficulty and name
- **Displays game rules**: Shows instructions text below

Program Code (Main Menu Tabs: Leaderboard Screen Code)

```
def setup_leaderboard_screen(self):
    self.frame_leaderboard = self._page()
    body = self.frame_leaderboard.body
    self._heading(body, "Leaderboards")
    tabs = tk.Frame(body, bg=self.colors["bg"])
    tabs.pack(fill="x", pady=(0, 18))
    self.leaderboard_buttons = {}
    for index, (mode, (difficulty, _, _)) in enumerate(MODES.items()):
        tabs.columnconfigure(index, weight=1, uniform="tabs")
        button = ActionButton(tabs, text=difficulty, variant="secondary", command=lambda m=mode: self.select_leaderboard(m))
        button.grid(row=0, column=index, sticky="ew", padx=(0, 8) if index < 2 else 0)
        self.leaderboard_buttons[mode] = button
    container = card(body, padding=20)
    container.pack(fill="x")
    self.lbl_board_title = label(container, "Word Scramble", size=19, bold=True)
    self.lbl_board_title.pack(anchor="w")
    self.lbl_board_personal = label(container, "", color=self.colors["muted"], wraplength=850, justify="left")
    self.lbl_board_personal.pack(anchor="w", fill="x", pady=(6, 18))
    table = tk.Frame(container, bg=self.colors["surface"])
    table.pack(fill="x")
    self.score_tree = ttk.Treeview(table, columns=("rank", "username", "score"), show="headings", selectmode="browse", height=8, takefocus=True)
    for column, heading, width, anchor in (("rank", "RANK", 80, "center"), ("username", "PLAYER", 350, "w"), ("score", "BEST SCORE", 110, "center")):
        self.score_tree.heading(column, text=heading)
        self.score_tree.column(column, width=width, minwidth=65, anchor=anchor, stretch=column == "username")
    scrollbar = ttk.Scrollbar(table, orient="vertical", command=self.score_tree.yview)
    self.score_tree.configure(yscrollcommand=scrollbar.set)
    scrollbar.pack(side="right", fill="y")
    self.score_tree.pack(side="left", fill="x", expand=True)
    self.score_tree.tag_configure("odd", background=self.colors["tile_empty"])
    self.score_tree.tag_configure("you", background=self.colors["soft"], foreground=self.colors["primary"], font=("Segoe UI", 11, "bold"))
    self.lbl_board_note = label(container, "", color=self.colors["muted"], wraplength=850, justify="left")
    self.lbl_board_note.pack(anchor="w", fill="x", pady=(15, 0))
    ActionButton(body, text="Play a Game", command=self.show_mode_selection).pack(anchor="w", pady=(18, 0))
```

Function:

- **Initializes Leaderboard Page**: Sets up leaderboard container and heading
- **Creates Mode Tabs**: Builds top game mode selection buttons
- **Builds Table Container**: Creates card layout for scores
- **Configures Treeview Table**: Sets up rank, player, and score columns
- **Adds Scrollbar & Styles**: Attaches scrollbar and row styling tags
- **Adds Play Button**: Creates shortcut button to start games

Program Code (Main Menu Tabs: Leaderboard Related Code)

```python
def show_current_leaderboard(self):
    if self.session is not None:
        self.leaderboard_mode.set(self.session.mode)
    self.show_leaderboard()

def select_leaderboard(self, mode):
    self.leaderboard_mode.set(mode)
    self.refresh_leaderboard()

def show_leaderboard(self):
    if self.active_user is not None and self._persist_score() and self.refresh_leaderboard():
        self.show_screen(self.frame_leaderboard)

def refresh_leaderboard(self):
    mode = self.leaderboard_mode.get()
    try:
        rows = self.db.leaderboard(mode)
    except (ValueError, sqlite3.Error) as error:
        self._error(error)
        return False
    self.lbl_board_title.config(text=MODES[mode][1])
    for key, button in self.leaderboard_buttons.items():
        button.config(variant="primary" if key == mode else "secondary")
    for item in self.score_tree.get_children():
        self.score_tree.delete(item)
    your_row = None
    for index, row in enumerate(rows):
        is_you = self.active_user is not None and row["username"] == self.active_user["username"]
        tags = ("you",) if is_you else (("odd",) if index % 2 else ())
        self.score_tree.insert("", "end", values=(f"#{row['rank']}",
                               row["username"] + ("  (you)" if is_you else ""), row["score"]), tags=tags)
        if is_you:
            your_row = row
    self.lbl_board_personal.config(
        text=f"Your rank: #{your_row['rank']}   ·   Your best: {your_row['score']} points" if your_row
        else "No personal score yet.")
    self.lbl_board_note.config(
        text=f"{len(rows)} {'player' if len(rows) == 1 else 'players'}  ·  Ties share a rank"
        if rows else "No scores yet.")
    return True
```

Function:

- **Shows current leaderboard**: Updates mode and displays view
- **Selects leaderboard mode**: Changes mode and refreshes board
- **Shows leaderboard screen**: Validates user and refreshes data
- **Refreshes leaderboard data**: Fetches database rows and updates UI

Program Code (Main Menu Tabs: Profile Screen Code Part 1)

```
def setup_edit_profile_screen(self):
    self.frame_edit_profile = self._page()
    self.frame_edit_profile.max_width = 900
    body = self.frame_edit_profile.body
    self._heading(body, "Profile")
    form = card(body, padding=24)
    form.pack(fill="x")
    self.entry_new_username, _ = self._field(form, "Username")
    self.entry_new_password, self.btn_show_new_password = self._field(form, "New password", password=True, hint="Leave blank to keep your current password.")
    self.profile_feedback = label(form, "", color=self.colors["muted"], wraplength=800, justify="left")
    buttons = tk.Frame(form, bg=self.colors["surface"])
    self.profile_actions = buttons
    buttons.pack(fill="x", pady=(20, 0))
    ActionButton(buttons, text="Save changes", command=self.save_profile).pack(side="left", padx=(0, 8))
    ActionButton(buttons, text="Cancel", variant="secondary", command=self.show_menu).pack(side="left")
    danger = card(body, padding=22, bg=self.colors["danger_bg"])
    danger.pack(fill="x", pady=(20, 0))
    label(danger, "Delete account", size=15, bold=True, color=self.colors["danger"]).pack(anchor="w")
    label(danger, "This permanently removes your account and all three leaderboard scores.", color=self.colors["muted"],
          wraplength=800, justify="left").pack(anchor="w", fill="x", pady=(7, 14))
    self.btn_delete_account = ActionButton(danger, text="Delete Account", variant="danger", command=self.delete_account)
    self.btn_delete_account.pack(anchor="w")
```

Function:

- **Initializes Profile Page**: Sets up edit profile container
- **Creates Form Card**: Builds card layout for inputs
- **Adds Input Fields**: Creates username and password fields
- **Adds Action Buttons**: Includes save and cancel buttons
- **Builds Danger Section**: Creates red card for account deletion
- **Adds Delete Button**: Includes confirmation button to delete account

Program Code (Main Menu Tabs: Profile Related Code)

```
def show_edit_profile(self):
    if self.active_user is None or not self._persist_score():
        return
    self.session = None
    self.entry_new_username.delete(0, tk.END)
    self.entry_new_username.insert(0, self.active_user["username"])
    self.entry_new_password.delete(0, tk.END)
    self._form_message(self.profile_feedback, "")
    self._hide_passwords()
    self.show_screen(self.frame_edit_profile)
```

Function:

- Validates user and saves scores
- Clears and populates username field
- Resets feedback and hides passwords
- Displays the edit profile screen

```
def save_profile(self):
    if self.active_user is None:
        return
    try:
        self.active_user = self.db.update_user(
            self.active_user["id"], self.entry_new_username.get().strip(),
            self.entry_new_password.get() or None)
    except (ValueError, sqlite3.Error) as error:
        self._form_message(self.profile_feedback, str(error), error=True)
        return
    self.entry_new_password.delete(0, tk.END)
    self._hide_passwords()
    self._form_message(self.notice, "Profile saved.")
    self.show_menu()
```

Function:

- Validates active user login state
- Updates database with new details

```
def delete_account(self):
    if self.active_user is None:
        return
    if not messagebox.askyesno(
            "Delete account?", "Permanently delete your account and all three leaderboard scores? This cannot be undone.",
            icon="warning", default="no", parent=self.root):
        return
    try:
        self.db.delete_user(self.active_user["id"])
    except sqlite3.Error as error:
        self._error(error)
        return
    self.session = None
    self.logout()
    self._form_message(self.auth_feedback, "Your account and scores have been deleted.")
```

Function:

- Checks if active user exists
- Prompts confirmation via message box
- Deletes account from database
- Resets session, logs out

Program Code (Main Menu Tab/Button: Logout Code)

```
def logout(self):
    if not self._persist_score():
        return
    self.active_user = None
    self.session = None
    for entry in (self.entry_username, self.entry_password, self.entry_new_username,
                  self.entry_new_password, self.entry_guess):
        entry.delete(0, tk.END)
    self._hide_passwords()
    self.set_auth_mode("login")
    self.show_screen(self.frame_login)
```

Function:

- **Saves pending user scores**: Save score before logging out
- **Clears active user session**: Sets user and session to None
- **Clears input fields**: Resets all text entry widgets
- **Hides password inputs**: Masks password fields for security
- **Resets screen**: Switches to login interface

Program Code (Game "Playing" Screen Code)

```
def setup_game_screen(self):
    self.frame_game = self._page()
    body = self.frame_game.body
    top = tk.Frame(body, bg=self.colors["bg"])
    top.pack(fill="x", pady=(0, 14))
    ActionButton(top, text="End run", variant="secondary", compact=True, command=self.finish_to_menu).pack(side="right")
    self.lbl_game_title = label(top, "", size=23, bold=True, wraplength=650, justify="left")
    self.lbl_game_title.pack(side="left", fill="x", expand=True)
    stats = card(body, padding=14)
    stats.pack(fill="x", pady=(0, 16))
    self.game_stat_labels = {}
    for index, (key, caption) in enumerate((("score", "Score"), ("best", "Best"), ("remaining", "Tries left"))):
        stats.columnconfigure(index, weight=1, uniform="game-stats")
        box = tk.Frame(stats, bg=self.colors["surface"])
        box.grid(row=0, column=index, sticky="nsew", padx=8)
        label(box, caption, size=10, bold=True, color=self.colors["muted"]).pack(anchor="w")
        value = label(box, "0", size=23, bold=True, color=self.colors["primary"] if key == "score" else self.colors["ink"])
        value.pack(anchor="w", pady=(4, 0))
        self.game_stat_labels[key] = value
    self.game_card = card(body, padding=20)
    self.game_card.pack(fill="x")
    self.puzzle_area = tk.Frame(self.game_card, bg=self.colors["surface"])
    self.answer_area = tk.Frame(self.game_card, bg=self.colors["surface"])
    self.game_card.bind("<Configure>", self._layout_game)
    self.lbl_question = label(self.puzzle_area, "", size=27, bold=True, wraplength=780, justify="center")
    self.wordle_panel = tk.Frame(self.puzzle_area, bg=self.colors["surface"])
    board = tk.Frame(self.wordle_panel, bg=self.colors["surface"])
    board.pack()
    self.wordle_tiles = []
    for row in range(5):
        cells = []
        for column in range(5):
            tile = tk.Label(board, text="", bg=self.colors["tile_empty"], fg=self.colors["ink"],
                            width=3, font=("Segoe UI", 17, "bold"), pady=3, relief="flat",
                            highlightthickness=1, highlightbackground=self.colors["border"])
            tile.grid(row=row, column=column, padx=3, pady=3)
            cells.append(tile)
        self.wordle_tiles.append(cells)
    legend = tk.Frame(self.wordle_panel, bg=self.colors["surface"])
    legend.pack(pady=(12, 9))
    self.legend_swatches = {}
    for status, text in (("correct", "Right place"), ("present", "Elsewhere"), ("absent", "Absent")):
        swatch = label(legend, "■", color=TILE_COLORS[status], size=12)
        swatch.pack(side="left", padx=(9, 3))
        self.legend_swatches[status] = swatch
        label(legend, text, size=10, color=self.colors["muted"]).pack(side="left")
    self.keyboard = {}
    self.keyboard_panel = tk.Frame(self.wordle_panel, bg=self.colors["surface"])
    self.keyboard_panel.pack(pady=(2, 3))
    for letters in ("qwertyuiop", "asdfghjkl", "zxcvbnm"):
        row_frame = tk.Frame(self.keyboard_panel, bg=self.colors["surface"])
        row_frame.pack(pady=2)
        for letter in letters:
            key = ActionButton(row_frame, text=letter.upper(), variant="secondary", compact=True,
                               command=lambda key=letter: self.type_letter(key), width=2, padx=3, pady=4,
                               font=("Segoe UI", 9, "bold"), takefocus=False)
            key.pack(side="left", padx=2)
            self.keyboard[letter] = key
    ActionButton(self.keyboard_panel, text="Delete letter", variant="ghost", compact=True,
                 command=self.backspace_guess, takefocus=False).pack(pady=(4, 0))
    self.lbl_rules = label(self.answer_area, "", color=self.colors["muted"], wraplength=750, justify="left")
    self.lbl_rules.pack(anchor="w", fill="x", pady=(0, 8))
    self.entry_guess, _ = self._field(self.answer_area, "Your answer", variable=self.guess_var)
    self.entry_guess.config(font=("Segoe UI", 16), justify="center")
    self.btn_submit = ActionButton(self.answer_area, text="Check answer", command=self.check_guess, state="disabled")
    self.btn_submit.pack(fill="x", pady=(14, 7))
    self.feedback_box = tk.Frame(self.answer_area, bg=self.colors["soft"], padx=14, pady=12)
    self.lbl_feedback = label(self.feedback_box, "", color=self.colors["primary"], wraplength=700, justify="left")
    self.lbl_feedback.pack(fill="x")
```

Main Function:

- **Initializes game container**: Sets layout
- **Builds stats display**: Shows scores
- **Creates puzzle grid**: Builds tiles
- **Adds keyboard panel**: Sets input keys
- **Configures submission controls**: Adds check button

Program Code (Game "End/Lose" Screen Code)

```python
def setup_end_screen(self):
    self.frame_end = self._page()
    self.frame_end.max_width = 820
    body = self.frame_end.body
    self.end_title = self._heading(body, "Game over")
    summary = card(body, padding=24)
    summary.pack(fill="x")
    self.lbl_end_mode = label(summary, "", size=12, bold=True, color=self.colors["primary"])
    self.lbl_end_mode.pack()
    self.lbl_final_score = label(summary, "0", size=60, bold=True)
    self.lbl_final_score.pack(pady=(6, 0))
    label(summary, "Final score", size=10, bold=True, color=self.colors["muted"]).pack()
    self.lbl_end_details = label(summary, "", color=self.colors["muted"], wraplength=800, justify="center")
    self.lbl_end_details.pack(fill="x", pady=18)
    self.btn_replay = ActionButton(summary, text="Play again", command=self.play_again)
    self.btn_replay.pack(fill="x", pady=(0, 8))
    ActionButton(summary, text="Choose another game", variant="secondary", command=self.show_mode_selection).pack(fill="x")
    ActionButton(body, text="View this leaderboard", variant="ghost", command=self.show_current_leaderboard).pack(pady=(14, 0))
```

Function:

- **Sets end page container**: Initializes layout page
- **Sets maximum page width**: Constrains container size
- **Adds game over heading**: Displays title text
- **Creates summary card container**: Builds results card
- **Adds mode and score labels**: Displays final game stats
- **Adds navigation buttons**: Creates replay and menu options

Program Code (Game "End/Lose" Related Code)

```
def show_end_screen(self, reason, saved=True):
    session = self.session
    difficulty, name, _ = MODES[session.mode]
    personal_best = session.score > self.starting_best and saved
    self.end_title.config(text="New best score" if personal_best else "Game over")
    self.lbl_end_mode.config(text=f"{difficulty.upper()}  /  {name}", fg=self.colors["mode_" + session.mode])
    self.lbl_final_score.config(text=str(session.score))
    save_note = f"Personal best: {self.best_score} points" if saved else "Score not saved yet. Choose an action below to retry."
    self.lbl_end_details.config(text=f"{reason}\nThe answer was {session.answer.upper()}.\n{save_note}")
    self.show_screen(self.frame_end)

def finish_to_menu(self):
    self.show_menu()

def play_again(self):
    if self.session is not None:
        self.start_game(self.session.mode)
```

`show_end_screen` Function:

- Updates end screen with final score and stats
- Checks for personal best and sets title text
- Displays correct answer and save status notes
- Switches active screen to end frame

`finish_to_menu` Function:

- Returns user back to the main menu

`play_again` Function:

- Restarts game session using current mode

Program Code (Game logic Code part 1)

```python
def __init__(self, mode, questions, allowed_words=None, rng=None):
    if mode not in MODES:
        raise ValueError("Choose a game mode: easy, medium, or hard.")
    self.mode = mode
    self._rng = rng if rng is not None else random.Random()
    self._questions = self._validate_questions(questions)
    self._allowed_words = {
        word.strip().lower()
        for word in (allowed_words if allowed_words is not None else ())
        if isinstance(word, str) and _is_word(word.strip(), 5)
    }
    self._allowed_words.update(question["answer"] for question in self._questions)
    self.score = 0
    self.mistakes = 0
    self.attempts = 0
    self.finished = False
    self.history = []
    self.current_question = None
    self.prompt = ""
    self.answer = ""
    self._bag = []
    self._last_question = None
    self._new_question()
```

Function:

- **Sets up random number generator**: Initializes RNG instance
- **Validates and stores questions**: Filters and loads questions
- **Builds allowed words set**: Processes valid dictionary words
- **Initializes game state**: Sets score, mistakes, and history
- **Spawns first question**: Triggers new question setup

Program Code (Game logic Code part 2)

```
def start_game(self, mode="easy"):
    if self.active_user is None or not self._persist_score():
        return
    try:
        session = GameSession(mode, self.db.get_questions(mode),
                              allowed_words=self.db.get_allowed_words() if mode == "medium" else None)
        best_score = self.db.record_score(self.active_user["id"], mode, 0)
    except (ValueError, sqlite3.Error) as error:
        self._error(error)
        return
    self.session = session
    self.best_score = self.starting_best = best_score
    difficulty, name, rules = MODES[mode]
    self.lbl_game_title.config(text=f"{name}  /  {difficulty}")
    self.lbl_rules.config(text=rules)
    self.btn_submit.config(text="Check word" if mode == "medium" else "Check answer")
    self._set_game_feedback("")
    self.guess_var.set("")
    self.render_game()
    self.show_screen(self.frame_game)
```

`start_game Function:`

- **Validate user**: Checks user and saves score
- **Initialize session**: Creates session with questions
- **Record score**: Initializes mode score
- **Handle error**: Displays error on failure
- **Update state**: Sets session and best scores
- **Set session**: Assigns session and updates best scores
- **Load mode**: Retrieves mode rules and metadata
- **Configure UI**: Updates title, rules, and button
- **Reset inputs**: Clears feedback and guess variable
- **Render game**: Draws layout and presents screen

Program Code (Game logic Code part 3)

```python
def _is_word(value, length=None):
    return (
        isinstance(value, str)
        and value.isascii()
        and value.isalpha()
        and (length is None or len(value) == length)
    )


def wordle_feedback(guess, answer):
    """Return five correct/present/absent marks, respecting repeated letters.

    Exact matches consume their answer letters first. A second pass assigns
    yellow marks only while unmatched copies of that letter remain.
    """
    if not isinstance(guess, str) or not isinstance(answer, str):
        raise ValueError("Wordle guesses and answers must contain five English letters.")
    guess, answer = guess.strip().lower(), answer.strip().lower()
    if not _is_word(guess, 5) or not _is_word(answer, 5):
        raise ValueError("Wordle guesses and answers must contain five English letters.")

    marks = ["absent"] * 5
    unmatched = Counter()
    for index, (guessed_letter, answer_letter) in enumerate(zip(guess, answer)):
        if guessed_letter == answer_letter:
            marks[index] = "correct"
        else:
            unmatched[answer_letter] += 1

    for index, letter in enumerate(guess):
        if marks[index] != "correct" and unmatched[letter] > 0:
            marks[index] = "present"
            unmatched[letter] -= 1
    return tuple(marks)
```

`_is_word` Function:

- Validates string properties and optional length

`wordle_feedback` Function:

- Evaluates guess against answer letter by letter
- Assigns correct, present, and absent marks

Program Code (Game logic Code part 4)

```python
def _validate_questions(self, questions):
    if questions is None:
        raise ValueError(f"No questions are available for {self.mode} mode.")
    validated = []
    seen = set()
    for index, question in enumerate(questions, start=1):
        if not isinstance(question, dict):
            raise ValueError(f"Question {index} must contain a question and answer.")
        answer = question.get("answer")
        if not isinstance(answer, str) or not answer.strip():
            raise ValueError(f"Question {index} has no answer.")
        answer = answer.strip().lower()
        sentence = question.get("question", "")
        if not isinstance(sentence, str):
            raise ValueError(f"Question {index} has invalid question text.")
        sentence = sentence.strip()
        if self.mode == "easy" and (not _is_word(answer) or len(set(answer)) < 2):
            raise ValueError(
                f"Easy question {index} needs an English word with at least "
                "two different letters so it can be scrambled."
            )
        if self.mode == "medium" and not _is_word(answer, 5):
            raise ValueError(f"Medium question {index} needs a five-letter English answer.")
        if self.mode == "hard" and "____" not in sentence:
            raise ValueError(f"Hard question {index} needs a sentence with a ____ blank.")
        # Repeated database rows must not make identical puzzles repeat.
        identity = (sentence if self.mode == "hard" else "", answer)
        if identity not in seen:
            normalized = dict(question)
            normalized.update(question=sentence, answer=answer)
            validated.append(normalized)
            seen.add(identity)
    if not validated:
        raise ValueError(f"No questions are available for {self.mode} mode.")
    return validated
```

Function:

- **Validates question format**: Ensures dictionary structure
- **Question data**: Strips and lowercases strings
- **Add easy mode rules**: Requires multi-letter word
- **Add medium/hard rules**: Checks word length and blank
- **Deduplicates question list**: Filters out duplicate items
- **Returns validated list**: Outputs processed questions

Program Code (Game logic Code part 5)

```python
def remaining_attempts(self):
    """Attempts left on this word, or remaining mistakes in easy/hard."""
    if self.mode == "medium":
        return max(0, 5 - self.attempts)
    return max(0, 3 - self.mistakes)

def _new_question(self):
    if not self._bag:
        self._bag = list(range(len(self._questions)))
        self._rng.shuffle(self._bag)
        # Avoid repeating the final puzzle of the previous shuffled bag.
        if len(self._bag) > 1 and self._bag[-1] == self._last_question:
            self._bag[0], self._bag[-1] = self._bag[-1], self._bag[0]
    index = self._bag.pop()
    self._last_question = index
    self.current_question = dict(self._questions[index])
    self.answer = self.current_question["answer"]
    self.attempts = 0
    self.history = []
    if self.mode == "easy":
        self.prompt = self._scramble(self.answer).upper()
    elif self.mode == "medium":
        self.prompt = "Guess the five-letter word. You have 5 tries."
    else:
        self.prompt = self.current_question["question"]

def _scramble(self, answer):
    letters = list(answer)
    for _ in range(8):
        self._rng.shuffle(letters)
        scrambled = "".join(letters)
        if scrambled != answer:
            return scrambled
    # Even an unlucky shuffle (or a deterministic test RNG) must produce
    # a different arrangement. Validation guarantees a distinct letter.
    for index in range(1, len(letters)):
        if letters[index] != letters[0]:
            letters[0], letters[index] = letters[index], letters[0]
            break
    return "".join(letters)
```

`remaining_attempts` Function:

- Calculates tries left based on mode

`_new_question` Function:

- Manages question bag and shuffling
- Sets up easy mode word scrambling

`_scramble` Function:

- Shuffles letters and ensures it changes order

Program Code (Game logic Code part 6)

```python
def submit(self, guess):
    """Submit a guess and return a status/message dictionary.

    Valid Wordle guesses also return a feedback tuple. Only a finished
    result reveals the answer. Invalid input never consumes a chance.
    """
    if self.finished:
        return self._finished_result()
    if not isinstance(guess, str) or not guess.strip():
        return {"status": "invalid", "message": "Enter an answer first."}
    guess = guess.strip().lower()
    feedback = None
    if self.mode == "medium":
        if not _is_word(guess, 5):
            return {"status": "invalid", "message": "Enter exactly five English letters."}
        if guess not in self._allowed_words:
            return {"status": "invalid", "message": "That word is not in the word list."}
        feedback = wordle_feedback(guess, self.answer)
        self.history.append((guess, feedback))
    self.attempts += 1

    if guess == self.answer:
        self.score += 1
        if self.mode == "hard":
            self.mistakes = 0
        self._new_question()
        result = {"status": "correct", "message": "Correct! +1 point. Try the next question."}
    else:
        if self.mode != "medium":
            self.mistakes += 1
        if self.remaining_attempts == 0:
            self.finished = True
            result = self._finished_result()
        else:
            remaining = self.remaining_attempts
            unit = "try" if remaining == 1 else "tries"
            result = {
                "status": "incorrect",
                "message": f"Not quite. {remaining} {unit} remaining.",
            }
    if feedback is not None:
        result["feedback"] = feedback
    return result

def _finished_result(self):
    return {
        "status": "finished",
        "message": f"Game over! Your final score is {self.score}.",
        "answer": self.answer,
    }
```

`Submit` Function:

- Validates guess status and input string
- Handles medium mode Wordle logic
- Checks correct answers and updates scores
- Tracks mistakes and remaining attempts

`_finished_result` Function:

- Returns final status and score

Program Code (Game logic Code part 7)

```python
def _persist_score(self):
    if self.session is None or self.active_user is None:
        return True
    try:
        self.best_score = self.db.record_score(self.active_user["id"], self.session.mode, self.session.score)
        return True
    except (ValueError, sqlite3.Error) as error:
        messagebox.showerror("Score could not be saved",
                             f"Your run is still open. Please retry before leaving.\n\n{error}", parent=self.root)
        return False

def show_menu(self):
    if self.active_user is None or not self._persist_score():
        return
    try:
        scores = self.db.get_high_scores(self.active_user["id"])
    except sqlite3.Error as error:
        self._error(error)
        return
    self.session = None
    self.lbl_welcome.config(text="Dashboard")
    for mode, value_label in self.best_value_labels.items():
        value_label.config(text=str(scores.get(mode, 0)))
    self.show_screen(self.frame_menu)
```

`_persist_score Function:`

- **Check session**: Exits if user or session is missing
- **Record score**: Saves score to database
- **Handle error**: Shows error popup on failure

`show_menu Function:`

- **Persist score**: Saves progress before switching
- **Fetch scores**: Loads high scores from database
- **Reset session**: Clears current session
- **Update UI**: Refreshes dashboard labels and shows menu

Program Code (Game logic Code part 8)

```python
def _layout_game(self, event):
    wide = event.width >= 820 and self.session is not None and self.session.mode == "medium"
    if wide == self._game_is_wide:
        return
    self._game_is_wide = wide
    self.puzzle_area.grid_forget()
    self.answer_area.grid_forget()
    self.game_card.columnconfigure(0, weight=1)
    self.game_card.columnconfigure(1, weight=1 if wide else 0)
    self.puzzle_area.grid(row=0, column=0, sticky="nsew",
                          padx=(0, 25) if wide else 0, pady=0 if wide else (0, 25))
    self.answer_area.grid(row=0 if wide else 1, column=1 if wide else 0, sticky="nsew")

def _set_game_feedback(self, text, status="info"):
    self._feedback_status = status
    foreground, background = {
        "info": (self.colors["primary"], self.colors["soft"]),
        "correct": (self.colors["success"], self.colors["success_bg"]),
        "incorrect": (self.colors["danger"], self.colors["danger_bg"]),
        "invalid": (self.colors["danger"], self.colors["danger_bg"]),
    }.get(status, (self.colors["muted"], self.colors["bg"]))
    self.feedback_box.config(bg=background)
    self.lbl_feedback.config(text=text, fg=foreground, bg=background)
    if text:
        self.feedback_box.pack(fill="x", pady=(18, 0))
    else:
        self.feedback_box.pack_forget()
```

`_layout_game Function`

- **Check layout**: Evaluates width threshold
- **Toggle state**: Updates layout state
- **Clear grid**: Removes existing areas
- **Set weights**: Configures grid columns
- **Reposition areas**: Places puzzle and answer grids

`_set_game_feedback Function`

- **Map colors**: Selects status colors
- **Update styles**: Applies text and styling
- **Display box**: Packs box if text exists
- **Hide box**: Hides box if empty

Program Code (Game logic Code part 9)

```python
def _on_guess_changed(self, *_args):
    can_submit = bool(self.guess_var.get().strip()) and self.session is not None and not self.session.finished
    self.btn_submit.config(state="normal" if can_submit else "disabled")
    self._render_pending_guess()

def _render_pending_guess(self):
    if self.session is None or self.session.mode != "medium" or self.session.finished:
        return
    row = len(self.session.history)
    if row >= 5:
        return
    pending = self.guess_var.get().strip().upper()[:5]
    for index, tile in enumerate(self.wordle_tiles[row]):
        tile.config(text=pending[index] if index < len(pending) else "", bg=self.colors["tile_empty"], fg=self.colors["ink"],
                    highlightbackground=self.colors["primary"] if index < len(pending) else self.colors["border"])

def type_letter(self, letter):
    if (self.current_frame is not self.frame_game or self.session is None
        or self.session.mode != "medium" or self.session.finished):
        return
    if self.entry_guess.selection_present():
        position = self.entry_guess.index(tk.SEL_FIRST)
        self.entry_guess.delete(tk.SEL_FIRST, tk.SEL_LAST)
    else:
        position = self.entry_guess.index(tk.INSERT)
    if len(self.entry_guess.get()) < 5:
        self.entry_guess.insert(position, letter.upper())
        self.entry_guess.icursor(position + 1)
    self.entry_guess.focus_set()
```

`_on_guess_changed` Function:

- **Check input**: Validates text presence
- **Toggle button**: Enables or disables submit button
- **Render pending**: Updates pending guess view

`_render_pending_guess` Function:

- **Validate state**: Checks medium mode and session
- **Get row**: Determines current history row
- **Format text**: Cleans and limits string
- **Update tiles**: Configures Wordle tile visuals

`type_letter` Function:

- **Validate state**: Checks game context
- **Handle selection**: Replaces selected text if active
- **Insert letter**: Adds uppercase letter if under limit
- **Focus input**: Sets focus to guess entry

Program Code (Game logic Code part 10)

```python
def backspace_guess(self):
    if self.current_frame is not self.frame_game or self.session is None or self.session.finished:
        return
    if self.entry_guess.selection_present():
        self.entry_guess.delete(tk.SEL_FIRST, tk.SEL_LAST)
    else:
        position = self.entry_guess.index(tk.INSERT)
        if position:
            self.entry_guess.delete(position - 1, position)
    self.entry_guess.focus_set()

def render_game(self):
    session = self.session
    if session is None:
        return
    self.game_stat_labels["score"].config(text=str(session.score))
    self.game_stat_labels["best"].config(text=str(max(self.best_score, session.score)))
    limit = 5 if session.mode == "medium" else 3
    self.game_stat_labels["remaining"].config(
        text=f"{session.remaining_attempts} / {limit}",
        fg=self.colors["danger"] if session.remaining_attempts == 1 else self.colors["ink"])
    self._game_is_wide = None
    self._layout_game(type("Layout", (), {"width": self.game_card.winfo_width()})())
    if session.mode == "medium":
        self.lbl_question.pack_forget()
        self.wordle_panel.pack()
        priorities = {"absent": 0, "present": 1, "correct": 2}
        key_statuses = {}
        for row, cells in enumerate(self.wordle_tiles):
            for column, tile in enumerate(cells):
                if row < len(session.history):
                    guess, statuses = session.history[row]
                    status = statuses[column]
                    tile.config(text=guess[column].upper(), bg=TILE_COLORS[status], fg="white",
                                highlightbackground=TILE_COLORS[status])
                    letter = guess[column]
                    if letter not in key_statuses or priorities[status] > priorities[key_statuses[letter]]:
                        key_statuses[letter] = status
                else:
                    tile.config(text="", bg=self.colors["tile_empty"], fg=self.colors["ink"], highlightbackground=self.colors["border"])
        for letter, key in self.keyboard.items():
            if letter in key_statuses:
                key.set_palette(TILE_COLORS[key_statuses[letter]])
            else:
                key.reset_palette()
        self._render_pending_guess()
    else:
        self.wordle_panel.pack_forget()
        question = "  ".join(session.prompt) if session.mode == "easy" else session.prompt
        self.lbl_question.config(text=question, font=("Segoe UI", 27 if session.mode == "easy" else 22, "bold"))
        self.lbl_question.pack(fill="x", pady=(12, 22))
```

`backspace_guess` Function:

- **Validate state**: Checks game context
- **Handle selection**: Deletes highlighted text
- **Delete character**: Removes previous character
- **Focus input**: Refocuses guess entry

`render_game` Function:

- **Validate session**: Checks active session
- **Update stats**: Refreshes score and attempts
- **Trigger layout**: Recalculates card dimensions
- **Render medium**: Updates tiles and keyboard
- **Render standard**: Formats prompt and label

Program Code (Game logic Code part 11)

```python
def check_guess(self):
    if self.current_frame is not self.frame_game or self.session is None or self.session.finished:
        return
    guess = self.entry_guess.get()
    result = self.session.submit(guess)
    status = result["status"]
    if status == "invalid":
        self._set_game_feedback(result["message"], "invalid")
        self.entry_guess.focus_set()
        return
    self.guess_var.set("")
    saved = self._persist_score() if status in ("correct", "finished") else True
    feedback = result["message"]
    if status == "correct":
        feedback = f"Correct: {guess.strip().upper()}  +1"
    elif self.session.mode == "medium" and result.get("feedback"):
        descriptions = {"correct": "right place", "present": "elsewhere", "absent": "absent"}
        feedback += "\n" + " · ".join(
            f"{letter.upper()}: {descriptions[mark]}" for letter, mark in zip(guess.strip(), result["feedback"]))
    self._set_game_feedback(feedback, status)
    self.render_game()
    if status == "finished":
        self.show_end_screen(result["message"], saved=saved)
    else:
        self.entry_guess.focus_set()

def show_end_screen(self, reason, saved=True):
    session = self.session
    difficulty, name, _ = MODES[session.mode]
    personal_best = session.score > self.starting_best and saved
    self.end_title.config(text="New best score" if personal_best else "Game over")
    self.lbl_end_mode.config(text=f"{difficulty.upper()}  /  {name}", fg=self.colors["mode_" + session.mode])
    self.lbl_final_score.config(text=str(session.score))
    save_note = f"Personal best: {self.best_score} points" if saved else "Score not saved yet. Choose an action below to retry."
    self.lbl_end_details.config(text=f"{reason}\nThe answer was {session.answer.upper()}.\n{save_note}")
    self.show_screen(self.frame_end)
```

`check_guess` Function:

- **Validate state**: Checks game context
- **Submit guess**: Evaluates user input
- **Handle invalid**: Sets error feedback
- **Persist score**: Saves completed score
- **Format feedback**: Builds result message
- **Update view**: Refreshes or shows end screen

`show_end_screen` Function:

- **Check score**: Determines personal best
- **Configure title**: Sets game status text
- **Update labels**: Displays mode and score details
- **Switch screen**: Presents end frame

Program Code (Game logic Code part 12)

```
def finish_to_menu(self):
    self.show_menu()

def play_again(self):
    if self.session is not None:
        self.start_game(self.session.mode)
```

`finish_to_menu` Function:

- **Show menu**: Returns to menu

`play_again` Function:

- **Check session**: Validates session
- **Restart**: Restarts current mode

Program Code (less Important Background Code)

```python
def _on_focus(self, event):
    if self.current_frame is not None and not self._closing:
        self.current_frame.scroll_to_widget(event.widget)
```

_on_focus Function:

- **Checks active frame**
- **Checks closing state**
- **Scrolls to focused widget**

```python
def _on_mousewheel(self, event, units=None):
    if self.current_frame is None or isinstance(event.widget, (ttk.Treeview, ttk.Scrollbar)):
        return
    ancestor = event.widget
    while ancestor is not None and ancestor is not self.current_frame:
        ancestor = getattr(ancestor, "master", None)
    if ancestor is None:
        return
    if units is None:
        if not event.delta:
            return
        units = -int(event.delta / 120) * 3 if abs(event.delta) >= 120 else (-1 if event.delta > 0 else 1)
    return self.current_frame.scroll(units)
```

_on_mousewheel Function:

- **Checks frame**: Validates widget
- **Finds ancestor**: Traces parent widget
- **Computes units**: Converts scroll delta
- **Scrolls view**: Updates frame position

Program Code (less Important Background Code)

```python
def _select_all(event):
    if isinstance(event.widget, tk.Entry):
        event.widget.selection_range(0, tk.END)
        event.widget.icursor(tk.END)
        return "break"

def _on_enter(self, event):
    if not isinstance(event.widget, tk.Entry):
        return
    if self.current_frame is self.frame_login:
        self.submit_auth()
    elif self.current_frame is self.frame_game:
        self.check_guess()
    elif self.current_frame is self.frame_edit_profile:
        self.save_profile()
    return "break"
```

_select_all Function:

- **Validate entry:** Checks widget type
- **Select range:** Highlights entry text
- **Move cursor:** Positions cursor at end

_on_enter Function:

- **Validate widget:** Checks entry type
- **Route action:** Triggers contextual submission based on active frame

Program Code (less Important Background Code)

```
def _form_message(self, target, text, error=False):
    target.config(text=text, fg=self.colors["danger"] if error else self.colors["success"])
    if text:
        before = {self.auth_feedback: self.btn_auth,
                  self.profile_feedback: self.profile_actions,
                  self.notice: self.btn_play}[target]
        target.pack(anchor="w", fill="x", pady=(10, 8), before=before)
    else:
        target.pack_forget()

def _error(self, error):
    messagebox.showerror("Unable to complete action", str(error), parent=self.root)
```

`_form_message Function:`

- **Set message**: Configures text and color
- **Map layout**: Finds reference widget
- **Display**: Packs widget if text exists
- **Hide**: Removes widget if empty

`_error Function:`

- **Show popup**: Displays system error dialog

```
def close(self):
    if not self._persist_score():
        return
    self._closing = True
    self.db.close()
    self.root.destroy()
```

close Function:

- **Persist score**: Saves current score
- **Set closing**: Updates closing flag
- **Close database**: Shuts down database connection
- **Destroy root**: Closes application window

UI code

(Define code for Screen function to use and customized what the user sees)

Program Code (UI Color Dictionary Code)

```
COLORS = {
    "bg": "#F4F6FB",
    "surface": "#FFFFFF",
    "ink": "#18243D",
    "muted": "#647087",
    "border": "#E1E6F0",
    "primary": "#4F46E5",
    "primary_hover": "#4338CA",
    "primary_pressed": "#3730A3",
    "on_primary": "#FFFFFF",
    "accent": "#4F46E5",
    "success": "#147D64",
    "success_bg": "#EAF7F1",
    "danger": "#C3394A",
    "danger_hover": "#AB2E3E",
    "danger_pressed": "#902535",
    "on_danger": "#FFFFFF",
    "danger_bg": "#FFF1F3",
    "navy": "#15243F",
    "soft": "#EEF0FF",
    "secondary_hover": "#E1E3FF",
    "secondary_pressed": "#D3D5F7",
    "ghost_hover": "#E9EDF5",
    "ghost_pressed": "#DBE1ED",
    "disabled_bg": "#E7EAF0",
    "disabled_fg": "#7A8598",
    "tile_empty": "#F6F7FC",
    "mode_easy": "#147D64",
    "mode_medium": "#4F46E5",
    "mode_hard": "#B43F65",
    "scroll": "#C7CEDD",
    "scroll_hover": "#A3AEC2",
}

THEMES = {
    "light": COLORS,
    "dark": {
        "bg": "#111827", "surface": "#1B2435", "ink": "#EDF2FA",
        "muted": "#B3BED2", "border": "#3E4C64",
        "primary": "#A79BFF", "primary_hover": "#B8AEFF",
        "primary_pressed": "#9788F2", "on_primary": "#171331",
        "accent": "#A79BFF", "success": "#6CD8B3",
        "success_bg": "#193B34", "danger": "#FF97A6",
        "danger_hover": "#FFADB9", "danger_pressed": "#EE8295",
        "on_danger": "#34121B", "danger_bg": "#3D2431",
        "navy": "#172039", "soft": "#302E50",
        "secondary_hover": "#3B365F", "secondary_pressed": "#453D70",
        "ghost_hover": "#2B374B", "ghost_pressed": "#35435A",
        "disabled_bg": "#303B4E", "disabled_fg": "#97A5BD",
        "tile_empty": "#232E43", "mode_easy": "#6CD8B3",
        "mode_medium": "#A79BFF", "mode_hard": "#F2A1BE",
        "scroll": "#52617A", "scroll_hover": "#7184A3",
    },
}
```

- **Defines `COLORS` dictionary**
- **Defines `THEMES` dictionary**
- **Configures dark mode**

Program Code (UI theme/layout Code)

```python
def colors_for(widget):
    """Read this window's palette without changing other application windows."""
    return getattr(widget.winfo_toplevel(), "_theme_colors", COLORS)
```

- **`colors_for` Function:**
- **Read palette**: Retrieves window theme colors

```python
def _layout_game(self, event):
    wide = event.width >= 820 and self.session is not None and self.session.mode == "medium"
    if wide == self._game_is_wide:
        return
    self._game_is_wide = wide
    self.puzzle_area.grid_forget()
    self.answer_area.grid_forget()
    self.game_card.columnconfigure(0, weight=1)
    self.game_card.columnconfigure(1, weight=1 if wide else 0)
    self.puzzle_area.grid(row=0, column=0, sticky="nsew",
                          padx=(0, 25) if wide else 0, pady=0 if wide else (0, 25))
    self.answer_area.grid(row=0 if wide else 1, column=1 if wide else 0, sticky="nsew")
```

- **`_layout_game` Function:**
- **Check width**: Evaluates window size and mode
- **Check state**: Exits if layout unchanged
- **Reset grid**: Clears current area positions
- **Configure columns**: Adjusts card weights
- **Position areas**: Grids puzzle and answer sections

Program Code (UI theme Code part 1)

```python
def apply_theme(root, theme_name):
    """Recolor existing controls in place, retaining form and keyboard state.

    Semantic roles are remembered after the first change so a light/dark round
    trip does not confuse colors shared by multiple roles. Explicit feedback
    palettes on ActionButton instances remain under the game renderer's control.
    """
    if theme_name not in THEMES:
        raise ValueError(f"Unknown theme: {theme_name}")
    old, new = colors_for(root), THEMES[theme_name]
    root._theme_colors = new
    root._theme_name = theme_name
    color_roles = {}
    for role, value in old.items():
        color_roles.setdefault(root.winfo_rgb(value), role)
    foreground_options = {
        "foreground", "activeforeground", "disabledforeground",
        "selectforeground", "insertbackground",
    }
    options = (
        "background", "foreground", "activebackground", "activeforeground",
        "disabledforeground", "highlightbackground", "highlightcolor",
        "insertbackground", "selectbackground", "selectforeground",
        "readonlybackground", "disabledbackground", "troughcolor",
    )
    white = root.winfo_rgb("white")

    def visit(widget):
        if isinstance(widget, ActionButton):
            widget.apply_theme()
            return
        if not isinstance(widget, ttk.Widget):
            roles = getattr(widget, "_theme_roles", {})
            updates = {}
            available = widget.keys()
            # Filled game tiles use white letters and stable feedback colors.
            fixed_fill = False
            if "foreground" in available and "background" in available:
                fixed_fill = (
                    widget.winfo_rgb(widget.cget("foreground")) == white
                    and widget.winfo_rgb(widget.cget("background")) != white
                )
            for option in options:
                if option not in available:
                    continue
                value = widget.cget(option)
                if not value:
                    continue
                try:
                    rgb = widget.winfo_rgb(value)
                except tk.TclError:
                    continue
                role = roles.get(option)
                if role and rgb != root.winfo_rgb(old[role]):
                    role = None  # A screen has assigned a different role.
                if role is None:
                    if option == "selectforeground" and rgb == root.winfo_rgb(old["on_primary"]):
                        role = "on_primary"
                    # White foregrounds on colored illustrations and tiles are
                    # intentionally constant; white surfaces still follow theme.
                    if role is None and option in foreground_options and rgb == white:
                        continue
                    if fixed_fill and option in ("background", "highlightbackground"):
                        continue
                    role = role or color_roles.get(rgb)
                if role:
                    roles[option] = role
                    updates[option] = new[role]
            widget._theme_roles = roles
            if updates:
                widget.configure(**updates)
        for child in widget.winfo_children():
            visit(child)

    visit(root)
    configure_styles(root)
    return new
```

`apply_theme` Function:

- **Validate theme**: Checks theme name validity
- **Load palette**: Sets up color dictionaries
- **Traverse widgets**: Recursively updates control styles
- **Configure styles**: Applies ttk styles and returns

Program Code (UI theme Code part 2)

```python
def configure_styles(root):
    """Apply a consistent font and neutral, readable ttk table controls."""
    colors = colors_for(root)
    root.option_add("*Font", ("Segoe UI", 11))
    root.option_add("*Entry.selectBackground", colors["primary"])
    root.option_add("*Entry.selectForeground", colors["on_primary"])
    root.option_add("*Listbox.selectBackground", colors["primary"])
    root.option_add("*Listbox.selectForeground", colors["on_primary"])
    style = ttk.Style(root)
    if "clam" in style.theme_names():
        style.theme_use("clam")
    style.configure(
        "Treeview", background=colors["surface"],
        fieldbackground=colors["surface"], foreground=colors["ink"],
        borderwidth=0, relief="flat", rowheight=42,
        font=("Segoe UI", 11),
    )
    style.configure(
        "Treeview.Heading", background=colors["bg"],
        foreground=colors["muted"], borderwidth=0, relief="flat",
        padding=(12, 12), font=("Segoe UI", 10, "bold"),
    )
    style.map(
        "Treeview", background=[("selected", colors["soft"])],
        foreground=[("selected", colors["primary"])],
    )
    style.map("Treeview.Heading", background=[("active", colors["ghost_hover"])])
    style.configure(
        "Vertical.TScrollbar", background=colors["scroll"],
        troughcolor=colors["bg"], bordercolor=colors["bg"],
        arrowcolor=colors["muted"], lightcolor=colors["scroll"],
        darkcolor=colors["scroll"], gripcount=0, borderwidth=0,
        arrowsize=12,
    )
    style.map("Vertical.TScrollbar", background=[("active", colors["scroll_hover"])])
    return style
```

`configure_styles` Function:

- **Read palette**: Retrieves window colors
- **Set defaults**: Configures global font and selection options
- **Apply theme**: Sets clam theme implementation
- **Style widgets**: Configures treeview and scrollbar components

Program Code (UI Layout Code)

```python
def label(parent, text="", size=11, bold=False, color=None,
          wraplength=None, **kwargs):
    """Create a label, optionally wrapping inside its parent as it resizes.

    Labels keep their natural width unless ``wraplength`` is supplied. An
    explicit positive value is a maximum that shrinks with the parent, keeping
    long headings and hints within a responsive page.
    Explicit widget options (including a background) take precedence.
    """
    options = {
        "text": text, "bg": parent.cget("bg"),
        "fg": color or colors_for(parent)["ink"],
        "font": ("Segoe UI", size, "bold" if bold else "normal"),
        "wraplength": wraplength or 0,
        "borderwidth": 0, "highlightthickness": 0,
    }
    options.update(kwargs)
    widget = tk.Label(parent, **options)
    if not wraplength or float(wraplength) <= 0:
        return widget
    maximum = float(wraplength)

    def fit(event):
        if event.width < 20 or not widget.winfo_exists():
            return
        try:
            parent_padding = parent.winfo_pixels(parent.cget("padx")) * 2
        except tk.TclError:
            parent_padding = 0
        available = max(40, event.width - parent_padding - 4)
        available = min(maximum, available)
        if float(widget.cget("wraplength")) != available:
            widget.configure(wraplength=available)

    binding = parent.bind("<Configure>", fit, add="+")

    def remove_binding(event):
        if event.widget is widget:
            try:
                parent.unbind("<Configure>", binding)
            except tk.TclError:
                pass  # The parent can be destroyed along with this label.

    widget.bind("<Destroy>", remove_binding, add="+")
    return widget
```

`label` Function:

- **Build options**: Configures widget properties and colors
- **Create label**: Instantiates text label widget
- **Set resizing**: Binds dynamic wrap-length updates
- **Bind cleanup**: Removes resize bindings on destroy

Program Code (UI Layout Code)

```python
def card(parent, padding=20, bg=None):
    """Return a flat card with generous internal spacing and a subtle edge."""
    colors = colors_for(parent)
    return tk.Frame(
        parent, bg=bg or colors["surface"], padx=padding, pady=padding,
        bd=0, highlightthickness=1, highlightbackground=colors["border"],
        highlightcolor=colors["border"],
    )
```

`card` Function:

- **Read palette**: Retrieves window colors
- **Create frame**: Builds styled container frame

Program Code (UI Class Code part 1)

```
class ActionButton(tk.Button):
    """A real Tk button with clear hover, pressed, focus, and disabled states."""
```

`ActionButton` Class:

- **Inherit `tk.Button`**: Extends standard widget
- **Manage states**: Handles hover, pressed, focus, and disabled states

Program Code ("ActionButton" UI theme Code)

```
_PALETTES = {
    "primary": ("primary", "primary_hover", "primary_pressed", "on_primary"),
    "secondary": ("soft", "secondary_hover", "secondary_pressed", "primary"),
    "ghost": (None, "ghost_hover", "ghost_pressed", "muted"),
    "danger": ("danger", "danger_hover", "danger_pressed", "on_danger"),
}
```

_PALETTES Variable:

- **Define palettes**: Configures button variant colors
- **Map roles**: Links hover, pressed, text states

Program Code ("ActionButton"UI Layout Code)

```
def __init__(self, parent, text="", command=None, variant="primary",
             compact=False, **kwargs):
    if variant not in self._PALETTES:
        raise ValueError(f"Unknown button variant: {variant}")
    self.variant = variant
    self._hovered = False
    self._pointer_down = False
    self._keyboard_down = None
    self._palette_override = None
    self._parent_bg = parent.cget("bg")
    options = {
        "text": text, "command": command, "relief": "flat",
        "borderwidth": 0, "highlightthickness": 2,
        "highlightbackground": self._parent_bg,
        "highlightcolor": colors_for(parent)["primary"],
        "padx": 13 if compact else 20,
        "pady": 7 if compact else 11,
        "font": ("Segoe UI", 10 if compact else 11, "bold"),
        "cursor": "hand2", "takefocus": True,
    }
    options.update(kwargs)
    super().__init__(parent, **options)
    self.bind("<Enter>", self._enter)
    self.bind("<Leave>", self._leave)
    self.bind("<ButtonPress-1>", self._mouse_press)
    self.bind("<ButtonRelease-1>", self._mouse_release)
    self.bind("<KeyPress-Return>", self._key_press)
    self.bind("<KeyRelease-Return>", self._key_release)
    self.bind("<KeyPress-KP_Enter>", self._key_press)
    self.bind("<KeyRelease-KP_Enter>", self._key_release)
    self.bind("<KeyPress-space>", self._key_press)
    self.bind("<KeyRelease-space>", self._key_release)
    self.bind("<Escape>", self._cancel_press)
    self.bind("<FocusOut>", self._cancel_press)
    self._render()
```

Function:

- **Validate variant**: Checks theme variant validity
- **Initialize state**: Sets up interaction flags
- **Configure options**: Sets button layout and styling properties
- **Bind events**: Registers mouse and keyboard listeners
- **Render button**: Draws initial appearance

Program Code ("ActionButton"UI Layout Code)

```
def configure(self, cnf=None, **kwargs):
    # Preserve Tk's querying API, including button.config("state").
    if cnf is not None and not isinstance(cnf, dict):
        return super().configure(cnf, **kwargs)
    updates = dict(cnf or {})
    updates.update(kwargs)
    if not updates:
        return super().configure()
    variant = updates.pop("variant", None)
    if variant is not None:
        if variant not in self._PALETTES:
            raise ValueError(f"Unknown button variant: {variant}")
        self.variant = variant
    result = super().configure(**updates) if updates else None
    if variant is not None or "state" in updates:
        if self.cget("state") == tk.DISABLED:
            self._pointer_down = False
            self._keyboard_down = None
        self._render()
    return result

config = configure
```

`configure` Function:

- **Parse arguments**: Handles configuration inputs
- **Update variant**: Validates and sets button style
- **Apply updates**: Configures super properties and state
- **Re-render**: Redraws button appearance

Program Code ("ActionButton"UI Layout Code)

```python
def set_palette(self, background, foreground="white", hover=None, pressed=None):
    """Keep semantic colors, such as Wordle feedback, across interactions.

    Feedback remains the same color on hover or press unless explicit
    alternatives are supplied. Disabled controls retain their neutral style
    and restore the semantic palette when they are enabled again.
    """
    palette = (background, hover or background, pressed or background, foreground)
    for color in palette:
        self.winfo_rgb(color)
    self._palette_override = palette
    self._render()
```

`set_palette` Function:

- **Build palette**: Sets custom color tuple
- **Validate colors**: Checks RGB values
- **Apply override**: Updates palette override and re-renders

Program Code ("ActionButton"UI Layout Code)

```
def reset_palette(self):
    """Return to the current variant's standard interaction colors."""
    self._palette_override = None
    self._render()
```

`reset_palette` Function:

- **Clear override:** Resets palette override
- **Re-render:** Redraws appearance

Program Code ("ActionButton"UI Layout Code)

```
def apply_theme(self):
    """Refresh colors while keeping the current interaction and game state."""
    self._parent_bg = self.master.cget("bg")
    tk.Button.configure(
        self, highlightbackground=self._parent_bg,
        highlightcolor=colors_for(self)["primary"],
    )
    self._render()
```

`apply_theme` **Function:**

- **Update background:** Retrieves and stores parent background color
- **Configure button:** Refreshes highlight background and primary color
- **Re-render:** Redraws button appearance

Program Code ("ActionButton"UI Layout Code)

```python
def _render(self):
    colors = colors_for(self)
    palette = self._palette_override or tuple(
        colors[role] if role else None for role in self._PALETTES[self.variant]
    )
    base, hover, pressed, foreground = palette
    background = base or self._parent_bg
    disabled = self.cget("state") == tk.DISABLED
    if disabled:
        background, foreground = colors["disabled_bg"], colors["disabled_fg"]
    elif self._keyboard_down or (self._pointer_down and self._hovered):
        background = pressed
    elif self._hovered:
        background = hover
    tk.Button.configure(
        self, bg=background, fg=foreground,
        activebackground=background, activeforeground=foreground,
        disabledforeground=colors["disabled_fg"], cursor="arrow" if disabled else "hand2",
    )
```

`_render` Function:

- **Resolve palette**: Selects override or variant colors
- **Determine state**: Checks disabled, pressed, or hovered status
- **Apply styles**: Configures button colors and cursor

Program Code ("ActionButton"UI Layout Code)

```
def _enter(self, _event):
    self._hovered = True
    self._render()

def _leave(self, _event):
    self._hovered = False
    self._render()

def _mouse_press(self, _event):
    if self.cget("state") != tk.DISABLED:
        self.focus_set()
        self._hovered = True
        self._pointer_down = True
        self._render()
    return "break"

def _mouse_release(self, event):
    invoke = self._pointer_down and self.cget("state") != tk.DISABLED
    invoke = invoke and 0 <= event.x < self.winfo_width() and 0 <= event.y < self.winfo_height()
    self._pointer_down = False
    self._render()
    if invoke:
        self.invoke()
    return "break"
```

`_enter` Function:

- **Hover**: Sets true

`_leave` Function:

- **Hover**: Sets false

`_mouse_press` Function:

- **Press**: Focuses and flags

`_mouse_release` Function:

- **Release**: Resets and invokes

Program Code ("ActionButton"UI Layout Code)

```
def _key_press(self, event):
    if self.cget("state") != tk.DISABLED and self._keyboard_down is None:
        self._keyboard_down = event.keysym
        self._render()
        # Return activates immediately; Space follows native release behavior.
        if event.keysym != "space":
            self.invoke()
    return "break"

def _key_release(self, event):
    invoke = self._keyboard_down == "space" and event.keysym == "space"
    self._keyboard_down = None
    self._render()
    if invoke and self.cget("state") != tk.DISABLED:
        self.invoke()
    return "break"

def _cancel_press(self, _event):
    self._keyboard_down = None
    self._pointer_down = False
    self._render()
```

`_key_press` Function:

- **Press**: Handles key down

`_key_release` Function:

- **Release**: Handles key up

`_cancel_press` Function:

- **Cancel**: Clears states

Program Code (UI class Code part 2)

```
class ScrollPage(tk.Frame):
    """A centered, width-limited page with scrolling when content needs it.
```

`ScrollPage` Class:

- **Inherit `tk.Frame`**: Extends standard container
- **Enable scrolling**: Centers content and handles overflow

Program Code ("ScrollPage" UI Layout Code)

```
def __init__(self, parent, max_width=1000, padding=28, **kwargs):
    kwargs.setdefault("bg", colors_for(parent)["bg"])
    super().__init__(parent, **kwargs)
    self.max_width = max_width
    self._layout_after = None
    self._scrollbar_visible = False
    self._content_height = 1
    self.canvas = tk.Canvas(
        self, bg=self.cget("bg"), bd=0, highlightthickness=0,
        yscrollincrement=24, takefocus=False,
    )
    self.scrollbar = ttk.Scrollbar(
        self, orient="vertical", command=self.canvas.yview,
        style="Vertical.TScrollbar", takefocus=False,
    )
    self.canvas.configure(yscrollcommand=self.scrollbar.set)
    self.canvas.pack(side="left", fill="both", expand=True)
    self.body = tk.Frame(
        self.canvas, bg=self.cget("bg"), padx=padding, pady=padding,
    )
    self._window = self.canvas.create_window(0, 0, anchor="nw", window=self.body)
    self.body.bind("<Configure>", self._schedule_layout)
    self.canvas.bind("<Configure>", self._schedule_layout)
    self.bind("<Destroy>", self._destroy_page, add="+")
```

Function:

- **Initialize base**: Sets up background and parent
- **Set properties**: Defines width and state flags
- **Setup canvas**: Creates scrollable canvas
- **Setup scrollbar**: Configures vertical scrollbar
- **Create body**: Adds inner frame
- **Bind events**: Registers listeners

Program Code ("ScrollPage"UI Layout Code)

```python
def _schedule_layout(self, _event=None):
    if self._layout_after is None:
        self._layout_after = self.after_idle(self._layout)

def _layout(self):
    self._layout_after = None
    if not self.winfo_exists():
        return
    width = max(1, self.canvas.winfo_width())
    height = max(1, self.canvas.winfo_height())
    body_width = min(self.max_width, width)
    self.canvas.itemconfigure(self._window, width=body_width)
    self.canvas.coords(self._window, max(0, (width - body_width) // 2), 0)
    self._content_height = self.body.winfo_reqheight()
    overflow = self._content_height > height + 1
    if overflow != self._scrollbar_visible:
        self._scrollbar_visible = overflow
        if overflow:
            self.scrollbar.pack(side="right", fill="y", before=self.canvas)
        else:
            self.scrollbar.pack_forget()
            self.canvas.yview_moveto(0)
    self.canvas.configure(scrollregion=(0, 0, width, max(height, self._content_height)))
```

`_schedule_layout` Function:

- **Schedule idle**: Queues layout update if pending
- **Category**: UI Layout

`_layout` Function:

- **Compute dimensions**: Calculates width and height
- **Center content**: Adjusts window size and position
- **Manage scrollbar**: Toggles visibility based on overflow
- **Update scrollregion**: Configures canvas scrolling area
- **Category**: UI Layout

Program Code ("ScrollPage"UI Layout Code)

```
def _destroy_page(self, event):
    if event.widget is self and self._layout_after is not None:
        self.after_cancel(self._layout_after)
        self._layout_after = None

def scroll(self, units):
    """Scroll by line units and consume a wheel event only when scrollable."""
    if self._content_height > self.canvas.winfo_height() + 1:
        self.canvas.yview_scroll(int(units), "units")
        return "break"
    return None
```

`_destroy_page` Function:

- **Cleanup**: Cancels pending layout tasks

`scroll` Function:

- **Scroll**: Moves view by units if scrollable

Program Code ("ScrollPage"UI Layout Code)

```python
def reset_scroll(self):
    self.canvas.yview_moveto(0)

def scroll_to_widget(self, widget):
    """Reveal a descendant reached with Tab without disturbing other pages."""
    ancestor = widget
    while ancestor is not None and ancestor is not self.body:
        ancestor = getattr(ancestor, "master", None)
    if ancestor is None or not widget.winfo_exists():
        return
    self.update_idletasks()
    if self._content_height <= self.canvas.winfo_height() + 1:
        return
    top = widget.winfo_rooty() - self.body.winfo_rooty()
    bottom = top + widget.winfo_height()
    visible_top = self.canvas.canvasy(0)
    visible_height = self.canvas.winfo_height()
    margin = 12
    target = None
    if top < visible_top + margin:
        target = top - margin
    elif bottom > visible_top + visible_height - margin:
        target = bottom + margin - visible_height
    if target is not None:
        self.canvas.yview_moveto(max(0, target) / self._content_height)
```

`reset_scroll` Function:

- **Reset position**: Moves canvas view to top

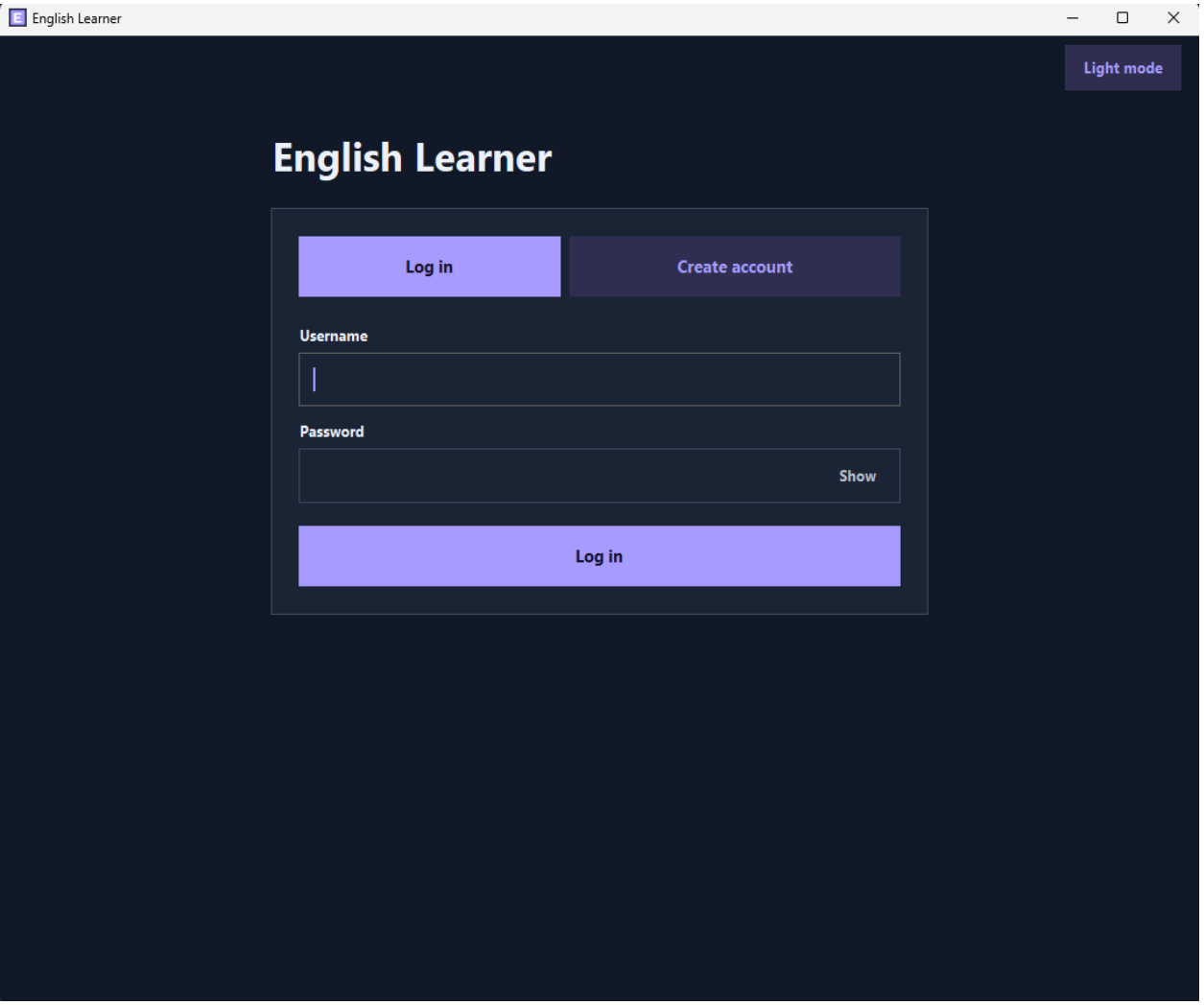
`scroll_to_widget` Function:

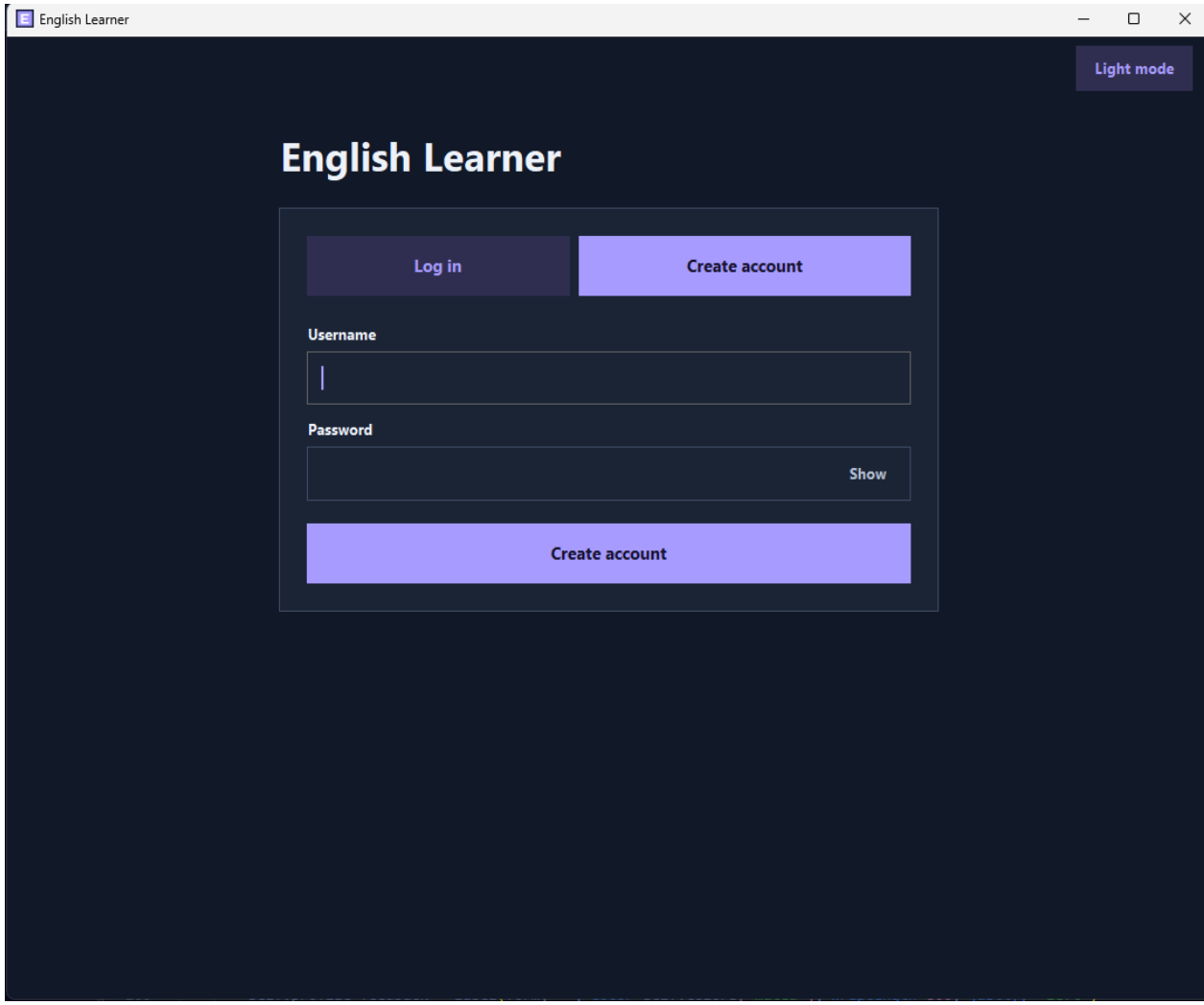
- **Validate ancestry**: Checks target widget
- **Calculate bounds**: Computes relative coordinates
- **Adjust view**: Scrolls canvas to reveal widget

From the Background code and UI code we will have a functioning UI app interface

Here Are The result
(UI Showcase)

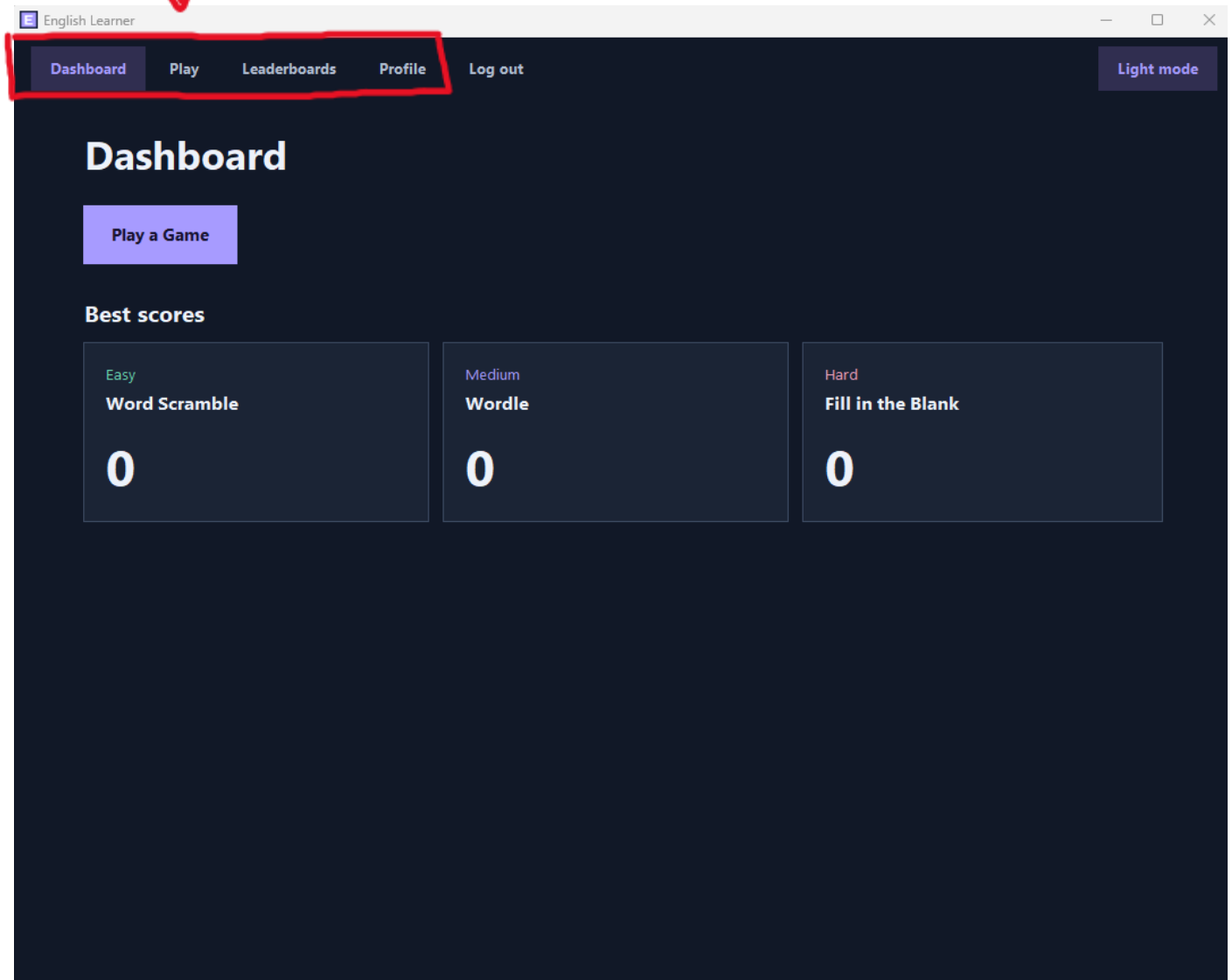
After runing the start up code, the gui will open the create/login interface





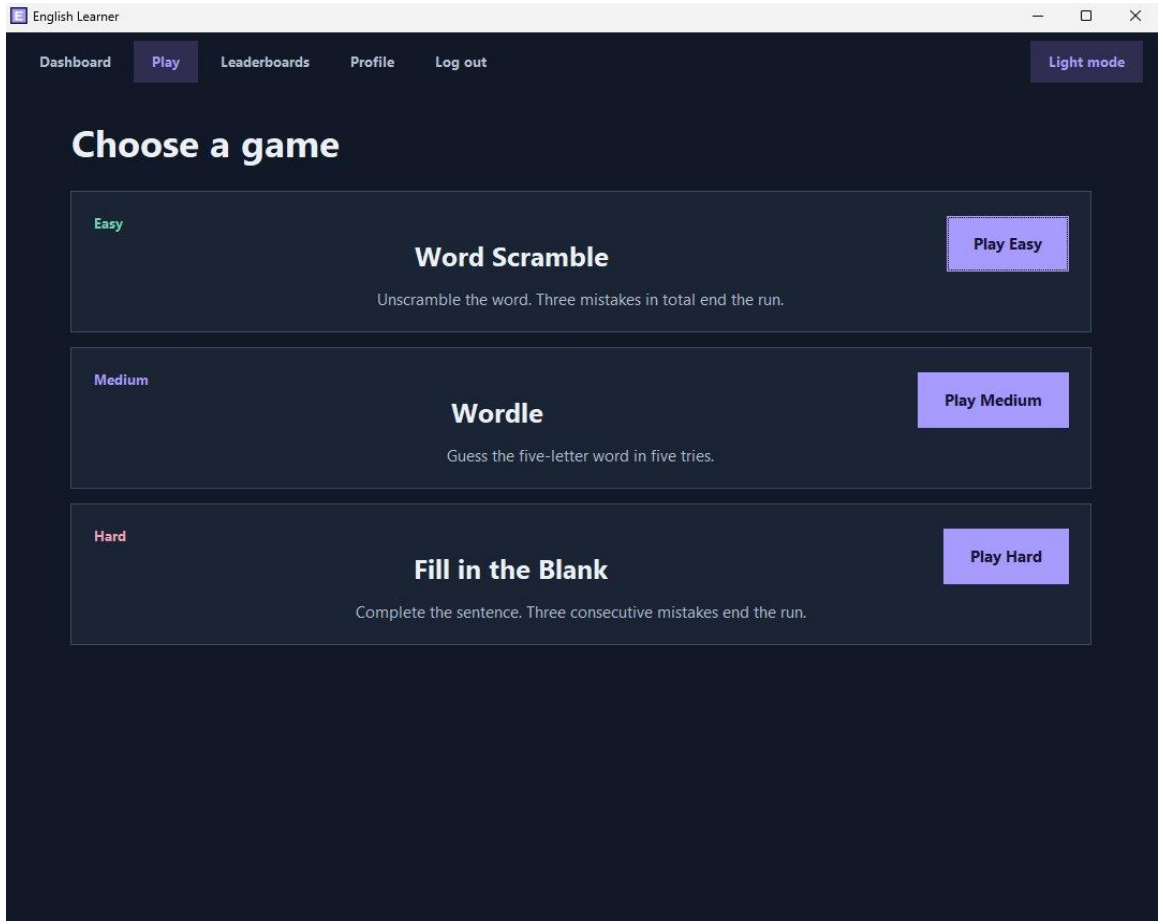
After logging in to the created account will change screen to the main menu

Which you can then select
any tab of ur choices

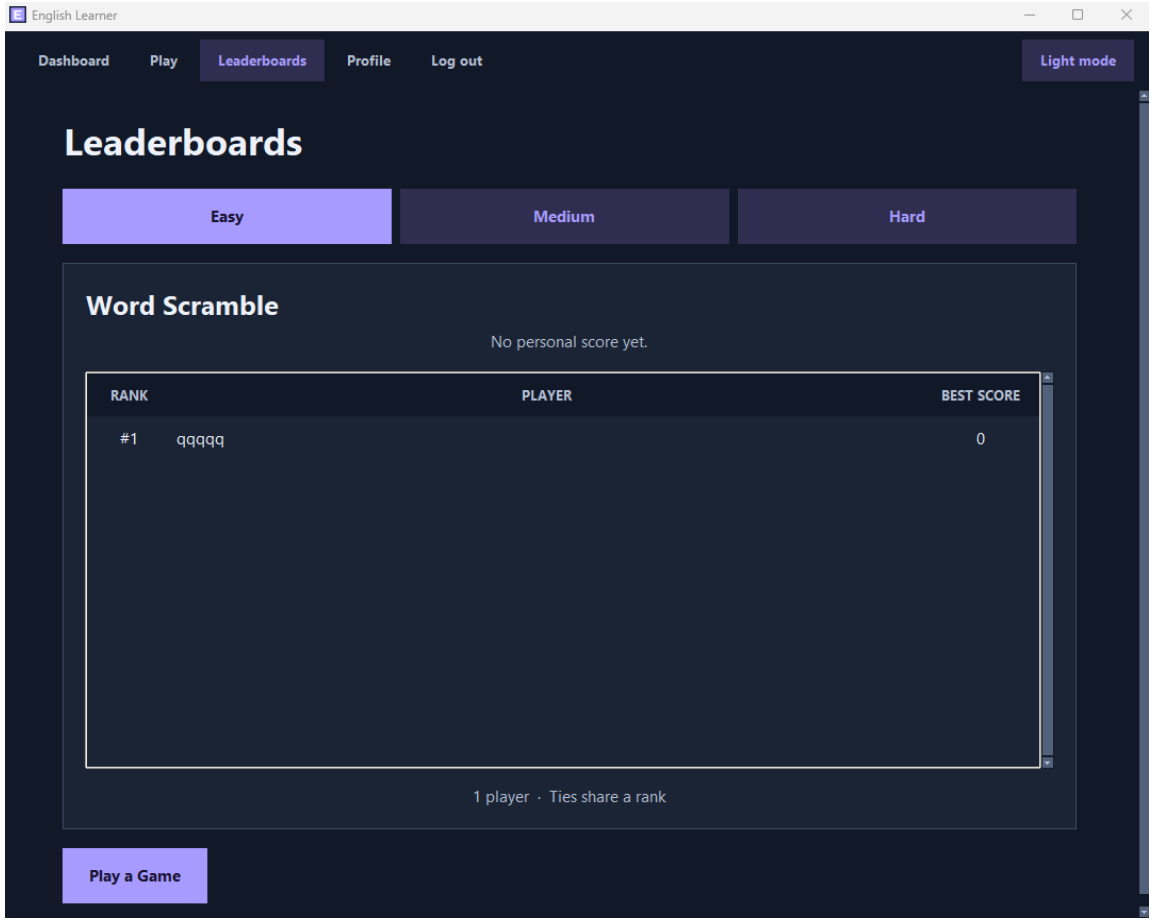


<---- Dashboard interface

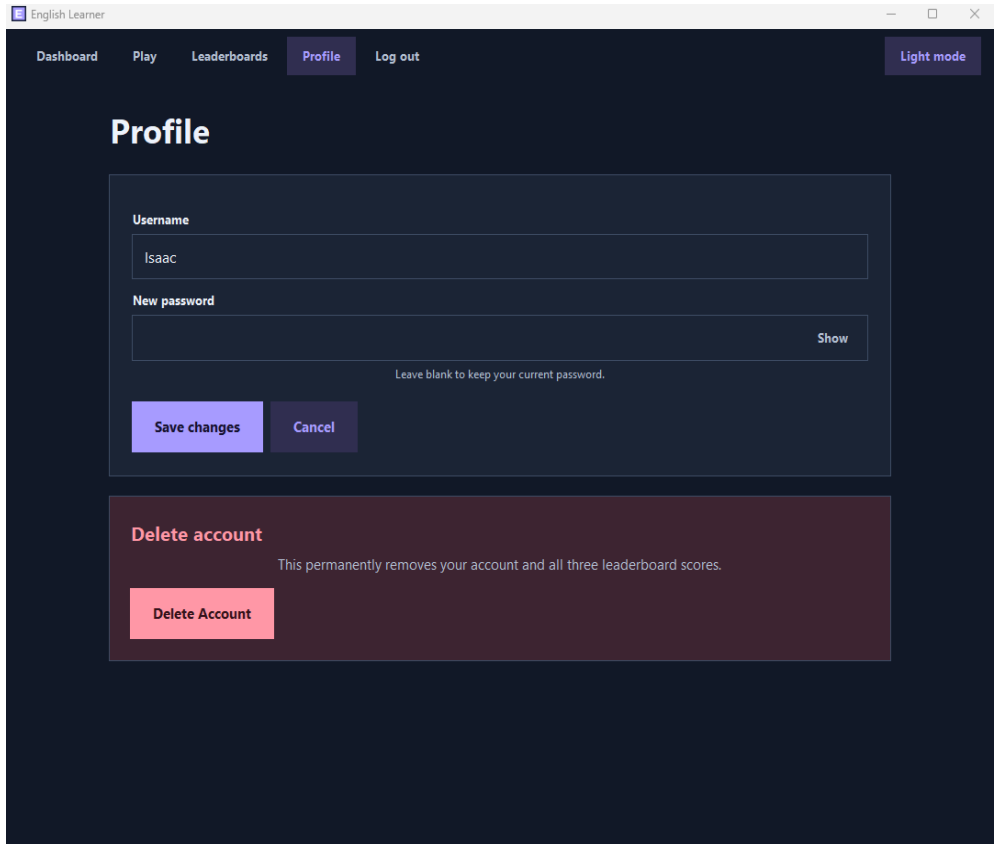
"Play" Interface



LeaderBoard Interface



Profile Interface



Database

For my data base, I have used sqLite database file to stored

- Question
- Answer
- UI Setting/theme
- User Information
- "Wordle Dictionary"

Permission

wordle_dictionary.sql

```
-- Wordle allowed-guess bootstrap dictionary. Import into SQLite once during setup.
-- Source: SCOWLv2 / English Speller Database, release 2026.02.25, size 70.
-- Project: https://wordlist.aspell.net/
-- Source file: https://raw.githubusercontent.com/en-wl/wordlist-diff/rel-2026.02.25/en_US-large.txt
-- Source file: https://raw.githubusercontent.com/en-wl/wordlist-diff/rel-2026.02.25/en_GB-large.txt
-- License source: https://raw.githubusercontent.com/en-wl/wordlist-diff/rel-2026.02.25/Copyright
-- Modification: union of US and GB lists; keep only exact lowercase ASCII [a-z]{5}; sort and deduplicate.
-- Entries: 7052. Allowed guesses only; answer words are seeded separately.
-- See THIRD_PARTY_NOTICES.md for source hashes and the complete applicable notice.
--
-- Copyright 2000-2026 by Kevin Atkinson
--
-- Permission to use, copy, modify, distribute, and sell any part of SCOWLv2, or
-- word lists created from it, is hereby granted without fee, provided that the
-- above copyright notice appears in all copies and that both the above
-- copyright notice and this notice appear in supporting documentation.  Kevin
-- Atkinson makes no representations about the suitability of this database for
-- any purpose.  It is provided "as is" without express or implied warranty.
--
-- SCOWL is derived from many sources, most of which are in the Public Domain.
-- Data from the Corpus of Contemporary American English (COCA) was also used.
--
-- All data from COCA comes from 3-gram data that is not freely available;
-- however, the usage is within the rights given by the NDA that was signed when
-- purchasing the data.  More information on COCA is available at
-- https://www.english-corpora.org/coca/.
--
-- The primary source of words for SCOWL comes from 12dicts and ENABLE2K.  Both
-- are in the Public Domain, but Alan Beale <biljir@pobox.com> deserves special
-- credit as he is the author of 12dicts and a major contributor to ENABLE2K.  In
-- addition, he gave me an incredible amount of feedback and created a number of
-- special lists in order to help improve the overall quality of SCOWL.
```

Program Code ("Define" Game Database Code)

```
SCRAMBLE_WORDS = tuple("""
apple beach bread brush chair clock cloud dance dream drink earth family
flower forest friend garden grape green happy heart honey house juice kitten
lemon light magic mango melon money moon mother mouse music night ocean orange
panda paper party peach pencil phone piano plant plate queen rabbit radio rain
river robot school sheep shirt shoes smile snake snow space spoon spring star
stone storm story sugar summer sunny table tiger today train tree truck turtle
uncle video voice water whale wheel white window winter woman world yellow zebra
animal basket bottle button camera candle carpet castle cheese cherry circle
coffee cookie cotton dragon finger island jacket kitchen ladder letter little
mirror monkey number orange pillow pocket potato purple rocket silver sister
soccer spider street teacher tomato travel turkey village violet wallet watch
""".split())
```

^^^^^^^^^^

vocabulary
(easy mode data)

Worldle solutions ------>
(medium mode data)

```
WORDLE_TARGETS = tuple("""
about above actor adult after again agree ahead alarm album alert alien alike
alive allow alone along alter among angel anger angle angry ankle apart apple
apply arena argue arise arrow aside avoid awake award aware awful badge baker
basic beach began begin below bench berry birth black blade blame bland blank
blast blend blind block blood bloom blown board boast bonus boost booth brain
brake brand brave bread break breed brick bride brief bring broad broke brown
brush build built bunch cabin cable camel canal candy carry catch cause chain
chair chalk charm chart chase cheap check cheek cheer chess chest chief child
chill choir claim class clean clear clerk click climb clock close cloth cloud
coach coast color comic coral couch could count court cover crack craft crane
crash crawl crazy cream creek crime crisp cross crowd crown cruel crush cycle
daily dairy dance death delay depth diary dirty doubt dough dozen draft drain
drama drank dream dress dried drink drive drove eagle early earth eaten eight
elder elect empty enemy enjoy enter entry equal error event every exact exist
extra faint fairy faith fancy fault feast fence fever field fifth fifty fight
final first flame flash fleet flesh float flock flood floor flour fluid flute
focus force found frame fresh front frost fruit funny giant given glass globe
glory glove going grace grade grain grand grant grape graph grasp grass grave
great green greet grind group grown guard guess guest guide guilt habit happy
harsh heart heavy hobby honey honor horse hotel house human humor hurry ideal
image imply index inner input issue ivory jelly jewel joint judge juice knife
knock known label labor large laser later laugh layer learn least leave legal
lemon level light limit linen liver local logic loose lover lower loyal lucky
lunch magic major maker mango maple march marry match maybe mayor medal media
melon mercy merit metal meter might minor mixed model money month moral motor
mount mouse mouth movie music nasty never newly night noble noise north novel
nurse occur ocean offer often olive onion opera orbit order organ other ought
outer owner paint panel panic paper party patch pause peace peach pearl pedal
penny phase phone photo piano piece pilot pitch place plain plane plant plate
point polar porch pound power press price pride prime print prize proof proud
prove pulse puppy queen quest quick quiet quite quote radio raise range rapid
ratio reach react ready rebel refer relax renew repay reply rider ridge right
rigid ripen risen river roast robot rocky rough round route royal ruler rural
salad sauce scale scare scarf scene scent score scrub sense serve seven shade
shake shall shame shape share shark sharp shave sheep sheer sheet shelf shell
shift shine shiny shirt shock shoot shore short shout shown sight since skill
skirt sleep slice slide slope small smart smile smoke snack snake solid solve
sorry sound south space spare speak speed spell spend spice spill spine split
spoke sport spray stack staff stage stair stake stand stare start state steam
steel steep steer stick still stock stone stood store storm story stove strip
stuck study stuff style sugar suite sunny super swear sweat sweep sweet swept
swift swing table taken taste teach teeth thank their theme there these thick
thief thing think third thorn those three throw thumb tiger tight timer tired
title toast today token tooth topic total touch tough tower track trade trail
train trait treat trend trial tribe trick tried troop truck truly trust truth
twice twist uncle under union unity until upper upset urban usage usual valid
value video virus visit vital vivid vocal voice waste watch water weave wedge
weigh weird whale wheat wheel where which while white whole whose wider woman
women world worry worse worst worth would wound write wrong wrote yacht yield
young youth zebra
""".split())
```

Program Code ("Define" Game Database Code)

Question and Answer Data --------> (hard mode data)

```
EXTRA_ALLOWED_WORDS = ("adieu", "audio", "fibre", "slate", "soare", "tears", "rates", "arose")
ALLOWED_WORDS = frozenset((*WORDLE_TARGETS, *EXTRA_ALLOWED_WORDS))

FILL_IN_BLANK_QUESTIONS = (
    ("A week has seven ____.", "days"),
    ("Water freezes into ____ when it gets cold enough.", "ice"),
    ("We use our ears to ____ sounds.", "hear"),
    ("A person who teaches students is a ____.", "teacher"),
    ("The opposite of hot is ____.", "cold"),
    ("Bees make sweet ____.", "honey"),
    ("A young dog is called a ____.", "puppy"),
    ("The season after winter is ____.", "spring"),
    ("There are twelve months in a ____.", "year"),
    ("You wear shoes on your ____.", "feet"),
    ("A person who bakes bread is a ____.", "baker"),
    ("We use our eyes to ____.", "see"),
    ("A triangle has three ____.", "sides"),
    ("The opposite of empty is ____.", "full"),
    ("Birds build ____ to lay their eggs in.", "nests"),
    ("A person who writes books is an ____.", "author"),
    ("The past tense of go is ____.", "went"),
    ("The past tense of eat is ____.", "ate"),
    ("The plural of child is ____.", "children"),
    ("The plural of tooth is ____.", "teeth"),
    ("Yesterday, I ____ a letter to my friend. (past tense of write)", "wrote"),
    ("She has ____ her homework. (past participle of finish)", "finished"),
    ("If I study hard, I will ____ the test. (opposite of fail)", "pass"),
    ("My brother is taller ____ me.", "than"),
    ("This is the ____ interesting book I have ever read. (superlative)", "most"),
    ("Neither Anna ____ Ben likes olives.", "nor"),
    ("I have lived here ____ 2020. (since or for)", "since"),
    ("We have waited here ____ two hours. (since or for)", "for"),
    ("They ____ playing football when it started to rain. (was or were)", "were"),
    ("There ____ five apples in the bowl. (is or are)", "are"),
    ("She ____ not enjoy loud music. (do or does)", "does"),
    ("Please speak ____; the baby is sleeping. (adverb of quiet)", "quietly"),
    ("The runner moved ____ toward the finish line. (adverb of quick)", "quickly"),
    ("A word with the opposite meaning is an ____.", "antonym"),
    ("A word with a similar meaning is a ____.", "synonym"),
    ("We borrowed books from the ____.", "library"),
    ("A doctor works in a ____ to treat patients. (starts with h)", "hospital"),
    ("The sun rises in the ____.", "east"),
    ("The sun sets in the ____.", "west"),
    ("A caterpillar grows into a ____.", "butterfly"),
    ("The ____ of France is Paris. (main city)", "capital"),
    ("We should wash our hands ____ eating. (before or during)", "before"),
    ("A person who studies science is a ____.", "scientist"),
    ("The opposite of generous is ____.", "selfish"),
    ("The comparative form of good is ____.", "better"),
    ("The superlative form of bad is ____.", "worst"),
    ("The past tense of bring is ____.", "brought"),
    ("The plural of mouse is ____.", "mice"),
    ("I look forward to ____ you tomorrow. (correct form of meet)", "meeting"),
    ("She is interested ____ learning new languages. (preposition)", "in"),
)
```

Program Code (Database Code)

```python
def __init__(self, path=None):
    self.path = Path(path).expanduser().resolve() if path not in (None, ":memory:") else path
    if self.path is None:
        self.path = DEFAULT_DATABASE
    self.conn = sqlite3.connect(str(self.path), timeout=10)
    self.conn.row_factory = sqlite3.Row
    self.conn.execute("PRAGMA foreign_keys = ON")
    self.conn.execute("PRAGMA busy_timeout = 10000")
    try:
        self._create_schema()
        self._seed()
        self._seed_dictionary()
        self.migration_report = self._empty_migration_report()
        if self.path != ":memory:":
            self.migration_report = self.import_legacy()
    except Exception:
        self.conn.close()
        raise
```

Function:

- **Resolve path**: Sets or defaults database path
- **Connect sqlite**: Opens connection with timeout
- **Configure connection**: Sets row factory and pragmas
- **Initialize schema**: Creates tables and seeds data

Program Code (Database Code)

```
def _create_schema(self):
    self.conn.executescript("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY,
            username TEXT NOT NULL COLLATE NOCASE UNIQUE,
            password_hash TEXT NOT NULL,
            password_salt TEXT NOT NULL,
            password_iterations INTEGER NOT NULL,
            created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
        );
        CREATE TABLE IF NOT EXISTS questions (
            id INTEGER PRIMARY KEY,
            mode TEXT NOT NULL CHECK (mode IN ('easy', 'medium', 'hard')),
            question TEXT NOT NULL,
            answer TEXT NOT NULL,
            UNIQUE (mode, question, answer)
        );
        CREATE TABLE IF NOT EXISTS high_scores (
            user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
            mode TEXT NOT NULL CHECK (mode IN ('easy', 'medium', 'hard')),
            score INTEGER NOT NULL CHECK (score >= 0),
            updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
            PRIMARY KEY (user_id, mode)
        );
        CREATE INDEX IF NOT EXISTS high_scores_mode_score
            ON high_scores(mode, score DESC);
        CREATE TABLE IF NOT EXISTS allowed_words (
            word TEXT PRIMARY KEY CHECK (length(word) = 5 AND word NOT GLOB '*[^a-z]*')
        );
        CREATE TABLE IF NOT EXISTS metadata (
            key TEXT PRIMARY KEY,
            value TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS legacy_scores (
            user_id INTEGER PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
            score INTEGER NOT NULL CHECK (score >= 0),
            imported_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
        );
        CREATE TABLE IF NOT EXISTS legacy_questions (
            id INTEGER PRIMARY KEY,
            question TEXT NOT NULL,
            answer TEXT NOT NULL,
            reason TEXT NOT NULL,
            UNIQUE (question, answer)
        );
    """)
```

_create_schema Function:
Create tables: Executes SQL script to set up schema

Program Code (Database Code)

```python
def _seed(self):
    with self.conn:
        # Claim initialization before inserting, making startup atomic even
        # if two application processes open a new database simultaneously.
        inserted = self.conn.execute(
            "INSERT OR IGNORE INTO metadata(key, value) VALUES ('seed_version', '1')"
        ).rowcount
        if not inserted:
            return
        questions = [("easy", "Unscramble the word.", word) for word in SCRAMBLE_WORDS]
        questions += [("medium", "Guess the five-letter word.", word) for word in WORDLE_TARGETS]
        questions += [("hard", question, answer) for question, answer in FILL_IN_BLANK_QUESTIONS]
        self.conn.executemany(
            "INSERT OR IGNORE INTO questions(mode, question, answer) VALUES (?, ?, ?)",
            questions,
        )
        self.conn.executemany("INSERT INTO allowed_words(word) VALUES (?)", ((w,) for w in sorted(ALLOWED_WORDS)))
```

_seed Function:

- **Ensure atomicity**: Uses connection context manager
- **Track version**: Inserts seed version metadata
- **Populate questions**: Combines scramble, wordle, and fill-in-blank data
- **Insert allowed words**: Bulk inserts sorted vocabulary

Program Code (Database Code)

```python
def _seed_dictionary(self):
    marker = "scowl_dictionary_version"
    if self.conn.execute("SELECT 1 FROM metadata WHERE key = ?", (marker,)).fetchone():
        return
    source = Path(__file__).resolve().with_name("wordle_dictionary.sql")
    if not source.is_file():
        return
    script = source.read_text(encoding="utf-8")
    with self.conn:
        if not self.conn.execute(
            "INSERT OR IGNORE INTO metadata(key, value) VALUES (?, '2026.02.25')", (marker,)
        ).rowcount:
            return
        # The bundled asset contains INSERT statements only. Execute each
        # in this transaction: executescript would commit the marker first.
        statement = ""
        for line in script.splitlines(keepends=True):
            statement += line
            if sqlite3.complete_statement(statement):
                self.conn.execute(statement)
                statement = ""
```

_seed_dictionary Function:

- **Check version marker**: Verifies metadata for existing dictionary version
- **Locate sql asset**: Finds bundled dictionary file
- **Track dictionary version**: Inserts version identifier into metadata table
- **Execute sql statements**: Parses and executes individual statements from script

Program Code (Database Code)

```python
def _validate_username(username):
    if not isinstance(username, str):
        raise ValueError("Please enter a username.")
    username = username.strip()
    if not username:
        raise ValueError("Please enter a username.")
    if len(username) > 32:
        raise ValueError("Your username must be 32 characters or fewer.")
    if not username.isprintable():
        raise ValueError("Your username cannot contain control characters.")
    return username
```

`_validate_username` Function:

- **Check name**: Makes sure the name is valid text

```python
def _validate_password(password):
    if not isinstance(password, str) or not password.strip():
        raise ValueError("Please enter a password.")
    if len(password) > 256:
        raise ValueError("Your password must be 256 characters or fewer.")
    return password
```

`_validate_password` Function:

- **Check password**: Makes sure the password is valid text

Program Code (Database Code)

```python
def _password_record(password):
    salt = secrets.token_hex(16)
    hashed = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), bytes.fromhex(salt), PASSWORD_ITERATIONS).hex()
    return hashed, salt, PASSWORD_ITERATIONS
```

_password_record Function:

- **Make password secure**: Creates a secure hash and salt

```python
def _validate_mode(mode):
    if mode not in MODES:
        raise ValueError("Please choose easy, medium, or hard mode.")
    return mode
```

_validate_mode Function:

- **Check game mode**: Makes sure the chosen mode is valid

```python
def _insert_user(self, username, password):
    hashed, salt, iterations = self._password_record(password)
    cursor = self.conn.execute(
        "INSERT INTO users(username, password_hash, password_salt, password_iterations) VALUES (?, ?, ?, ?)",
        (username, hashed, salt, iterations),
    )
    return {"id": cursor.lastrowid, "username": username}
```

_insert_user Function:

- **Hash password**: Creates secure password details
- **Save user**: Adds the new user to the database
- **Return info**: Gives back the new user ID and name

```python
def create_user(self, username, password):
    username = self._validate_username(username)
    password = self._validate_password(password)
    try:
        with self.conn:
            return self._insert_user(username, password)
    except sqlite3.IntegrityError as exc:
        raise ValueError("That username is already taken.") from exc
```

create_user Function:

- **Check input:** Validates the username and password
- **Add user:** Saves the new user to the database
- **Handle duplicates:** Prevents duplicate usernames with an error

Program Code (Database Code)

```python
def authenticate(self, username, password):
    if not isinstance(username, str) or not isinstance(password, str) or len(password) > 256:
        return None
    row = self.conn.execute("SELECT * FROM users WHERE username = ?", (username.strip(),)).fetchone()
    if row is None:
        return None
    actual = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), bytes.fromhex(row["password_salt"]), row["password_iterations"]
    ).hex()
    if hmac.compare_digest(actual, row["password_hash"]):
        return {"id": row["id"], "username": row["username"]}
    return None
```

authenticate Function:

- **Check input**: Validates username and password
- **Find user**: Looks up the account in the database
- **Verify password**: Checks the password securely

```python
def update_user(self, user_id, username, password=None):
    username = self._validate_username(username)
    if password not in (None, ""):
        self._validate_password(password)
    try:
        with self.conn:
            if password not in (None, ""):
                hashed, salt, iterations = self._password_record(password)
                cursor = self.conn.execute(
                    "UPDATE users SET username = ?, password_hash = ?, password_salt = ?, password_iterations = ? WHERE id = ?",
                    (username, hashed, salt, iterations, user_id),
                )
            else:
                cursor = self.conn.execute("UPDATE users SET username = ? WHERE id = ?", (username, user_id))
            if cursor.rowcount != 1:
                raise ValueError("This account no longer exists. Please log in again.")
    except sqlite3.IntegrityError as exc:
        raise ValueError("That username is already taken.") from exc
    return {"id": user_id, "username": username}
```

update_user Function:

- **Check input**: Validates the new name and password
- **Update database**: Changes account info
- **Handle errors**: Prevents duplicate name

Program Code (Database Code)

```
def delete_user(self, user_id):
    with self.conn:
        self.conn.execute("DELETE FROM users WHERE id = ?", (user_id,))

def get_high_scores(self, user_id):
    result = dict.fromkeys(MODES, 0)
    rows = self.conn.execute("SELECT mode, score FROM high_scores WHERE user_id = ?", (user_id,))
    result.update((row["mode"], row["score"]) for row in rows)
    return result
```

delete_user Function:

- **Remove user**: Deletes a user account

get_high_scores Function:

- **Fetch scores**: Gets user high scores for all modes

```
def get_theme(self) -> str:
    """Return this installation's theme, including before anyone logs in."""
    row = self.conn.execute("SELECT value FROM metadata WHERE key = ?", ("ui_theme",)).fetchone()
    return row["value"] if row is not None and row["value"] in ("light", "dark") else "light"

def set_theme(self, theme: str) -> None:
    """Save a supported appearance without changing accounts or scores."""
    if theme not in ("light", "dark"):
        raise ValueError("Please choose the light or dark theme.")
    with self.conn:
        self.conn.execute("""
            INSERT INTO metadata(key, value) VALUES (?, ?)
            ON CONFLICT(key) DO UPDATE SET value = excluded.value
        """, ("ui_theme", theme))
```

get_theme Function:

- **Get theme**: Looks up the app theme, defaulting to light

set_theme Function:

- **Save theme**: Validates and saves the chosen theme

Program Code (Database Code)

```python
def record_score(self, user_id, mode, score):
    self._validate_mode(mode)
    if type(score) is not int or not 0 <= score <= 2**63 - 1:
        raise ValueError("A score must be a non-negative whole number.")
    try:
        with self.conn:
            self.conn.execute("""
                INSERT INTO high_scores(user_id, mode, score) VALUES (?, ?, ?)
                ON CONFLICT(user_id, mode) DO UPDATE SET
                    score = MAX(high_scores.score, excluded.score),
                    updated_at = CASE WHEN excluded.score > high_scores.score
                        THEN CURRENT_TIMESTAMP ELSE high_scores.updated_at END
            """, (user_id, mode, score))
            row = self.conn.execute("SELECT score FROM high_scores WHERE user_id = ? AND mode = ?", (user_id, mode)).fetchone()
            return row["score"]
    except sqlite3.IntegrityError as exc:
        raise ValueError("This account no longer exists. Please log in again.") from exc
```

record_score Function:

- **Save score**: Validates and saves the high score

```python
def leaderboard(self, mode):
    self._validate_mode(mode)
    rows = self.conn.execute("""
        SELECT u.username, h.score FROM high_scores h
        JOIN users u ON u.id = h.user_id WHERE h.mode = ?
        ORDER BY h.score DESC, u.username COLLATE NOCASE, u.username
    """, (mode,)).fetchall()
    result, previous_score, rank = [], None, 0
    for position, row in enumerate(rows, 1):
        if row["score"] != previous_score:
            rank = position
        result.append({"rank": rank, "username": row["username"], "score": row["score"]})
        previous_score = row["score"]
    return result
```

leaderboard Function:

- **Get top scores**: Shows ranked scores for a game

Program Code (Database Code)

```python
def get_questions(self, mode):
    self._validate_mode(mode)
    return [dict(row) for row in self.conn.execute(
        "SELECT id, question, answer FROM questions WHERE mode = ? ORDER BY id", (mode,)
    )]

def get_allowed_words(self):
    return {row[0] for row in self.conn.execute(
        "SELECT word FROM allowed_words UNION SELECT answer FROM questions WHERE mode = 'medium'"
    )}
```

get_questions Function:

- **Get questions**: Loads questions for a chosen game mode

get_allowed_words Function:

- **Get word list**: Gathers valid words for the game

Program Code (Migration Database Code)

```
def _empty_migration_report():
    return {
        "users_imported": 0, "users_skipped": 0, "legacy_scores_imported": 0,
        "questions_imported": 0, "questions_skipped": 0, "questions_archived": 0,
        "warnings": [],
    }
```

_empty_migration_report Function:

- **Blank report**: Gives back a zeroed-out migration

Program Code (Migration Database Code)

```python
def import_legacy(self, directory=None):
    """Import the original comma/pipe format, once per legacy file.

    Old cumulative scores are archived in ``legacy_scores`` because ten
    points per answer across all sessions cannot be compared with a new
    single-session high score. Non-fillable questions are preserved in
    ``legacy_questions``. No plaintext passwords are stored in SQLite.
    """
    if directory is None:
        directory = DEFAULT_DATABASE.parent if self.path == ":memory:" else self.path.parent
    directory = Path(directory)
    report = self._empty_migration_report()
    sources = (("user-data.txt", self._import_user_lines), ("questions.txt", self._import_question_lines))
    for filename, importer in sources:
        source = directory / filename
        marker = "legacy_import:" + filename
        if not source.is_file() or self.conn.execute("SELECT 1 FROM metadata WHERE key = ?", (marker,)).fetchone():
            continue
        try:
            # UTF-8 with BOM is accepted; preserve old Windows text files too.
            raw = source.read_bytes()
            try:
                content = raw.decode("utf-8-sig")
            except UnicodeDecodeError:
                content = raw.decode("cp1252")
                report["warnings"].append(f"{filename}: imported using Windows text encoding.")
        except (OSError, UnicodeError) as exc:
            report["warnings"].append(f"Could not import {filename}: {exc}")
            continue
        with self.conn:
            # Claim before importing and commit together so concurrent
            # launches cannot replay the same input after a deletion.
            if not self.conn.execute("INSERT OR IGNORE INTO metadata(key, value) VALUES (?, ?)", (marker, str(source.resolve()))).rowcount:
                continue
            importer(content.splitlines(), report)
    self.migration_report = report
    return report
```

import_legacy Function:

- **Import old data**: Loads old user and question files

Program Code (Migration Database Code)

```python
def _import_user_lines(self, lines, report):
    for number, line in enumerate(lines, 1):
        if not line.strip():
            continue
        parts = line.strip().split(",")
        try:
            if len(parts) != 3:
                raise ValueError("expected username,password,score")
            username = self._validate_username(parts[0])
            password = self._validate_password(parts[1])
            if not parts[2].isascii() or not parts[2].isdigit():
                raise ValueError("score must be a non-negative whole number")
            score = int(parts[2])
            if score > 2**63 - 1:
                raise ValueError("score is too large")
            if self.conn.execute("SELECT 1 FROM users WHERE username = ?", (username,)).fetchone():
                raise ValueError("duplicate username")
            user = self._insert_user(username, password)
            self.conn.execute("INSERT INTO legacy_scores(user_id, score) VALUES (?, ?)", (user["id"], score))
            report["users_imported"] += 1
            report["legacy_scores_imported"] += 1
        except ValueError as exc:
            report["users_skipped"] += 1
            report["warnings"].append(f"user-data.txt line {number}: skipped ({exc}).")
```

_import_user_lines Function:

- **Import user data**: Reads and adds old user accounts

Program Code (Migration Database Code)

```python
def _import_question_lines(self, lines, report):
    for number, line in enumerate(lines, 1):
        if not line.strip():
            continue
        parts = line.strip().split("|")
        if len(parts) != 2 or not all(part.strip() for part in parts):
            report["questions_skipped"] += 1
            report["warnings"].append(f"questions.txt line {number}: skipped malformed question|answer.")
            continue
        original, answer = (part.strip() for part in parts)
        answer = answer.lower()
        question, reason = re.sub(r"_{2,}", "____", original), None
        if not re.fullmatch(r"[a-z]+", answer):
            reason = "Answer must be a single English word."
        elif not re.search(r"_{2,}", question):
            pattern = re.compile(r"(?<!\w)" + re.escape(answer) + r"(?!\w)", re.IGNORECASE)
            question, count = pattern.subn("____", question, count=1)
            if not count:
                reason = "Question has no blank and does not contain its answer."
        if reason:
            inserted = self.conn.execute(
                "INSERT OR IGNORE INTO legacy_questions(question, answer, reason) VALUES (?, ?, ?)",
                (original, answer, reason),
            ).rowcount
            report["questions_archived"] += inserted
            report["questions_skipped"] += 1 - inserted
            report["warnings"].append(f"questions.txt line {number}: archived for review. {reason}")
            continue
        inserted = self.conn.execute(
            "INSERT OR IGNORE INTO questions(mode, question, answer) VALUES ('hard', ?, ?)",
            (question, answer),
        ).rowcount
        report["questions_imported"] += inserted
        report["questions_skipped"] += 1 - inserted
        if not inserted:
            report["warnings"].append(f"questions.txt line {number}: skipped duplicate question.")
```

_import_question_lines Function:

- **Import questions**: Reads and adds old question files

Program Code (Database Code)

```
def close(self):
    self.conn.close()

def __enter__(self):
    return self

def __exit__(self, exc_type, exc_value, traceback):
    self.close()
```

close Function:

- **Close connection**: Closes the database connection

__enter__ Function:

- **Enter context**: Returns self for context manager

__exit__ Function:

- **Exit context**: Closes connection on exit

Use Of Variables

- **`MODES`**: Defines available game modes (`easy`, `medium`, `hard`) across the application.
- **`DEFAULT_DATABASE`**: Specifies the default SQLite database path (`english_learning.db`).
- **`PASSWORD_ITERATIONS`**: Sets the iteration count (600,000) for secure password hashing.
- **`SCRAMBLE_WORDS`**: Curated tuple of words used to seed easy scramble questions.
- **`WORDLE_TARGETS`**: Curated tuple of words used to seed medium Wordle target answers.
- **`ALLOWED_WORDS`**: Curated set of permitted Wordle guesses.
- **`FILL_IN_BLANK_QUESTIONS`**: Curated collection of sentences and answers for hard mode.
- **`self.conn`**: Handles the active SQLite database connection.
- **`self.db`**: Manages database persistence operations.
- **`self.active_user`**: Stores the profile details (`id` and `username`) of the currently logged-in user.
- **`self.session`**: Tracks active gameplay state, scores, attempts, and guess history via `GameSession`.
- **`self.theme_name`**: Stores the active UI theme appearance (`light` or `dark`).
- **`self.colors`**: Holds the active UI color palette dictionary.
- **`self.frames`**: Manages the list of UI screen layout pages.
- **`self.current_frame`**: Tracks the currently visible UI screen frame.
- **`self.migration_report`**: Tracks legacy file import counts and warnings.

Program Code (Database Code)

System Chart (Relationship of Modules)

Important Variables and Data Structure

Important Functions (Main Program) (Description and Flowchart / Program Codes)

Important Functions (Loading User Data) (Description and Flowchart / Program Codes)

Screen Capture of Sample Program Outcomes